

ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L4: The processor (2)

Computer Structure

Bachelor in Computer Science and Engineering

Bachelor in Applied Mathematics and Computing

Dual Bachelor in Computer Science and Engineering and Business Administration



Contents

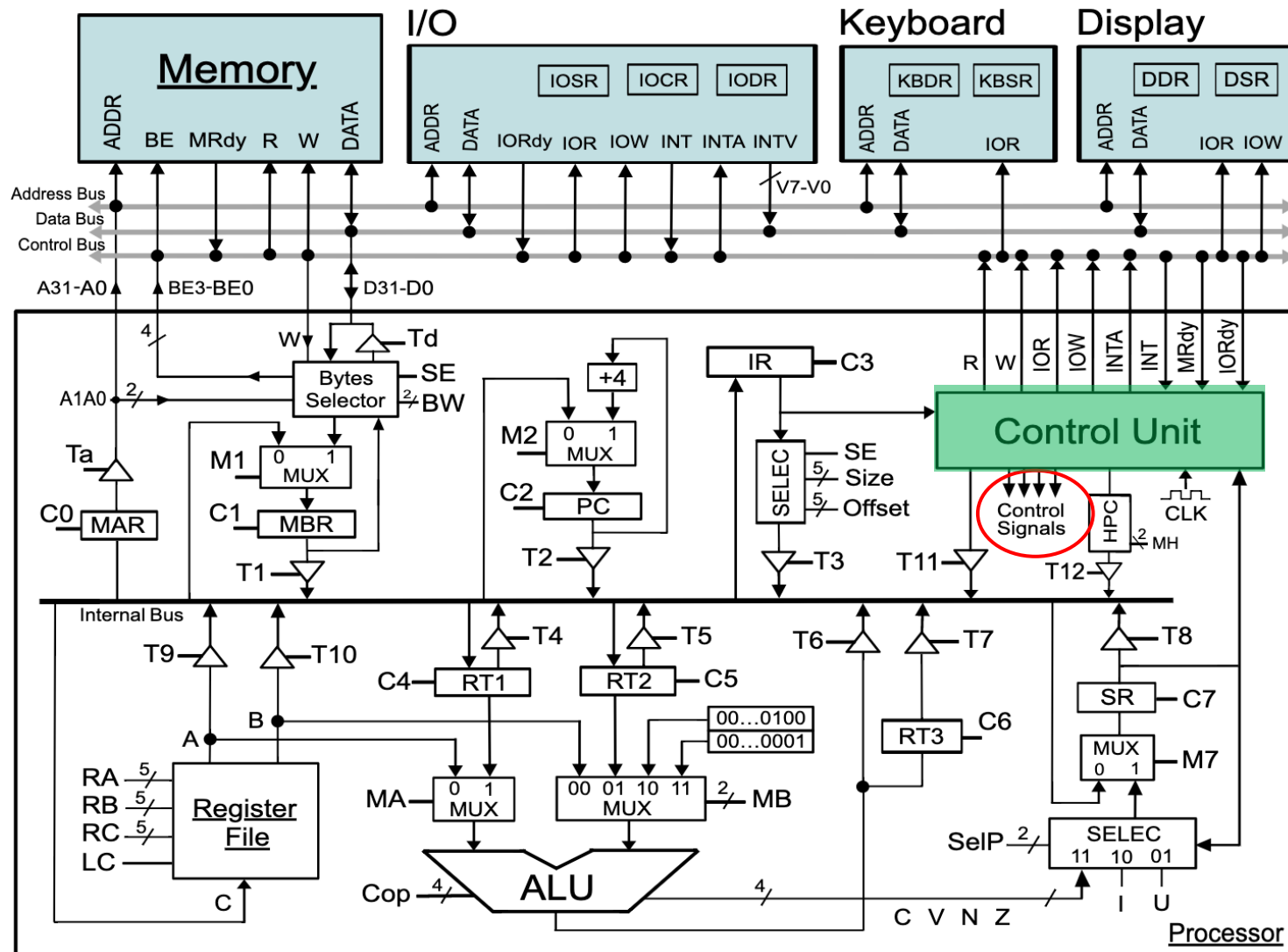
1. Computer elements
2. Processor organization
3. The control unit
4. Execution of instructions
5. Control unit design
 - a) Tasks in the design of a control unit
 - b) Microprogram control unit
 - c) Control unit in WepSIM
 - d) Example of a microprogrammed instruction set
6. Execution modes
7. Interrupts
8. Booting a computer
9. Performance and parallelism

Reminder



Control unit

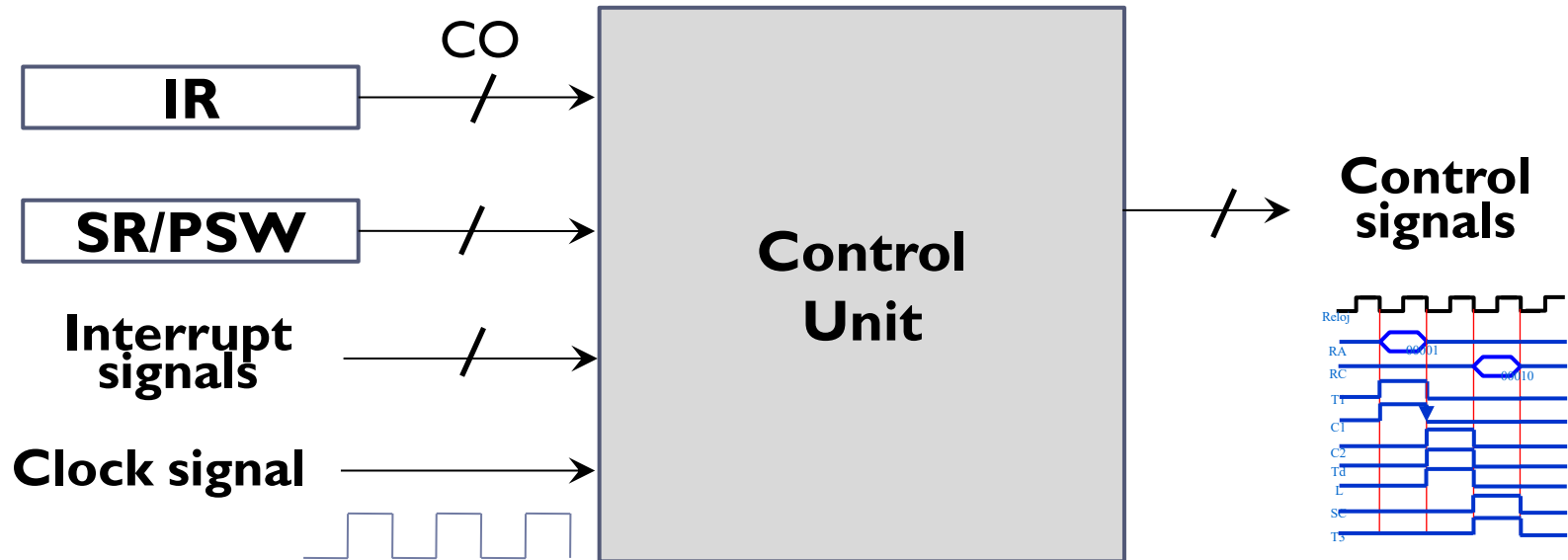
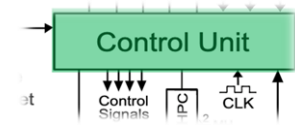
Reminder



Contents

1. Computer elements
2. Processor organization
3. The control unit
4. Execution of instructions
5. Control unit design
 - a) Tasks in the design of a control unit
 - b) Microprogram control unit
 - c) Control unit in WepSIM
 - d) Example of a microprogrammed instruction set
6. Execution modes
7. Interrupts
8. Booting a computer
9. Performance and parallelism

Control unit



- ▶ Every **control signal** is **function** of the values of:
 - ▶ The content of the **IR**
 - ▶ The content of **SR**
 - ▶ The **period of time (clock)**

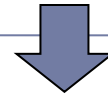
Control unit design

- For each machine instruction:

1. Define the behavior using RTL (register transfer language) for every clock cycle
2. Translate the behavior to values of each control signal at each clock cycle
3. Design a circuit that generates the value of each control signal at each clock cycle

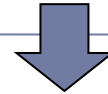
Instruction

mv R0 R1

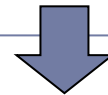
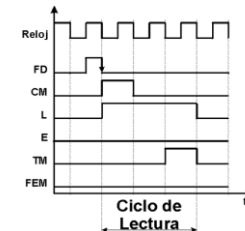


Sequence of **elementary operations**

1. $IR \leftarrow [PC]$
2. $PC++$
3. decode
4. $R0 \leftarrow R1$



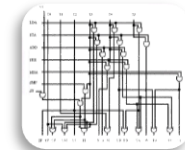
Sequence of **control signals** for each elementary operation



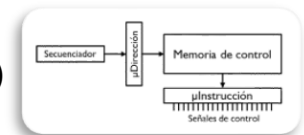
Circuit that generates signals:

- a) Hardwired control
- b) Microprogrammed control

a)



b)



Example

- ▶ Design of a control unit for a set of 4 machine instructions.
- ▶ Instructions to consider:
 - ▶ `add Rd, Rf:` `Rd <- Rd + Rf`
 - ▶ `lw Rd, dir:` `Rd <- MP[dir]`
 - ▶ `sw Rf, dir:` `MP[dir] <- Rf`
 - ▶ `bz R, dir:` `if (R==0) PC<- dir`

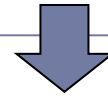
Control unit design

- For each machine instruction:

1. Define the behavior using RTL (register transfer language) for every clock cycle
2. Translate the behavior to values of each control signal at each clock cycle
3. Design a circuit that generates the value of each control signal at each clock cycle

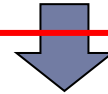
Instruction

mv R0 R1

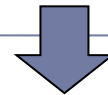
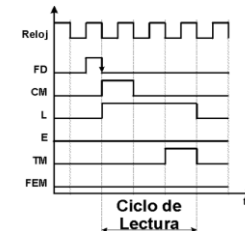


Sequence of **elementary operations**

1. IR <- [PC]
2. PC++
3. decode
4. R0 <- R1



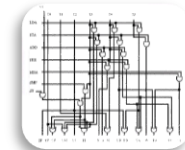
Sequence of **control signals** for each elementary operation



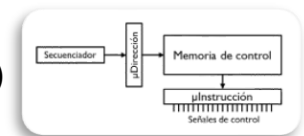
Circuit that generates signals:

- a) Hardwired control
- b) Microprogrammed control

a)

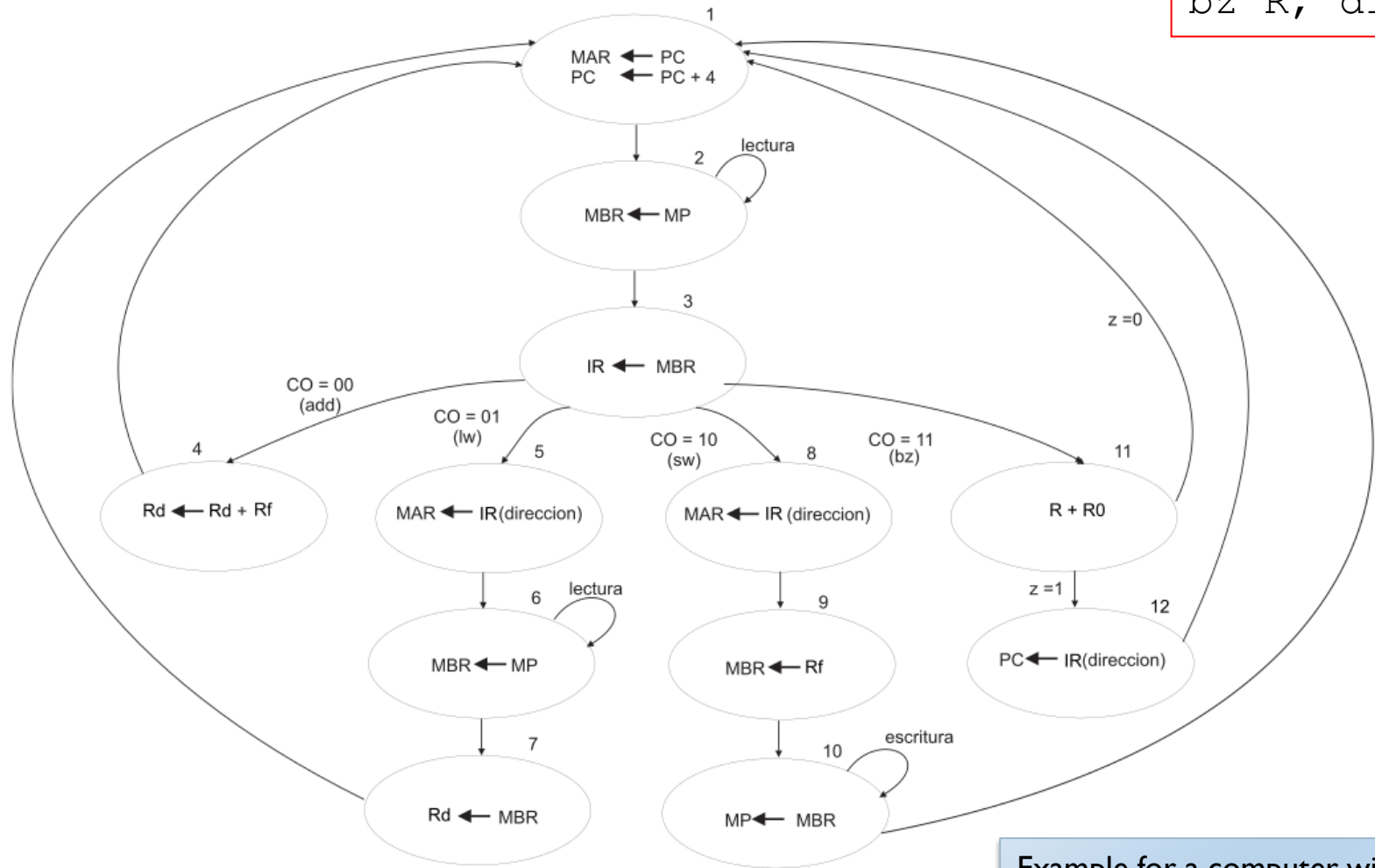


b)



State machine for the example

```
add rd, rf  
lw rd, dir  
sw Rf, dir  
bz R, dir
```



Example for a computer with only 4 machine instructions

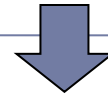
Control unit design

- For each machine instruction:

1. Define the behavior using RTL (register transfer language) for every clock cycle
2. Translate the behavior to values of each control signal at each clock cycle
3. Design a circuit that generates the value of each control signal at each clock cycle

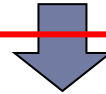
Instruction

mv R0 R1

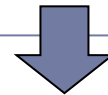
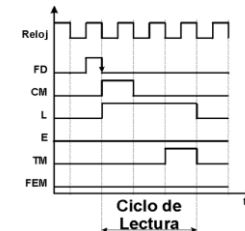


Sequence of **elementary operations**

1. $IR \leftarrow [PC]$
2. $PC++$
3. decode
4. $R0 \leftarrow R1$



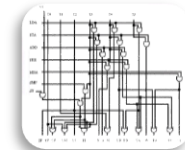
Sequence of **control signals** for each elementary operation



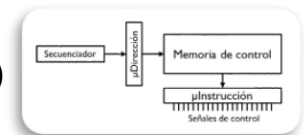
Circuit that generates signals:

- a) Hardwired control
- b) Microprogrammed control

a)

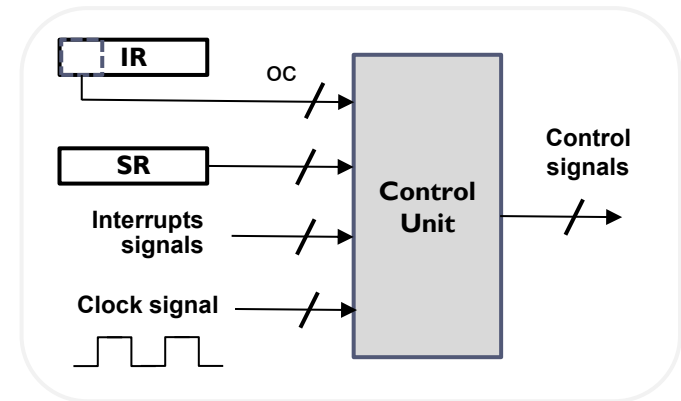
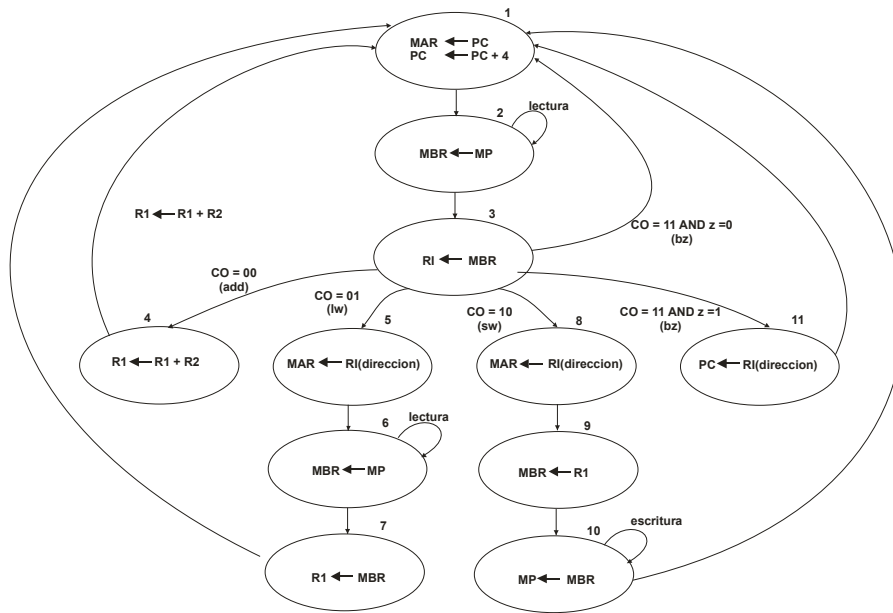


b)



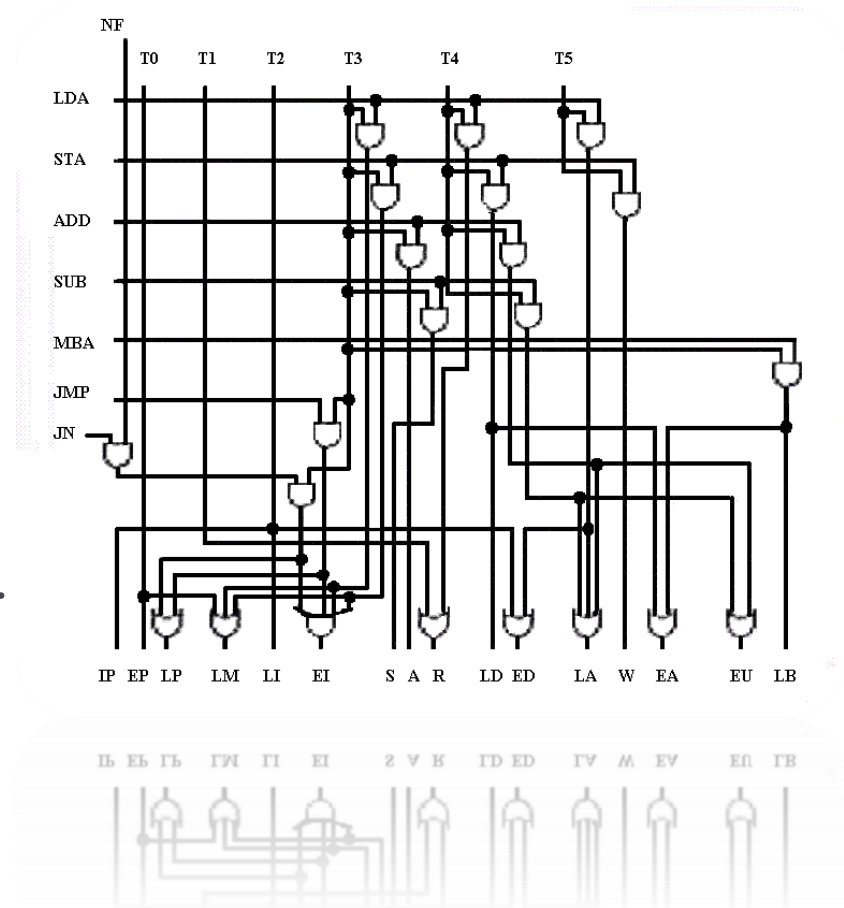
Control techniques

- ▶ **Two techniques** to design and build the control unit:
 - a) **Relay logic**
 - b) **Programmable logic (microprogrammed)**



Control Unit: relay logic

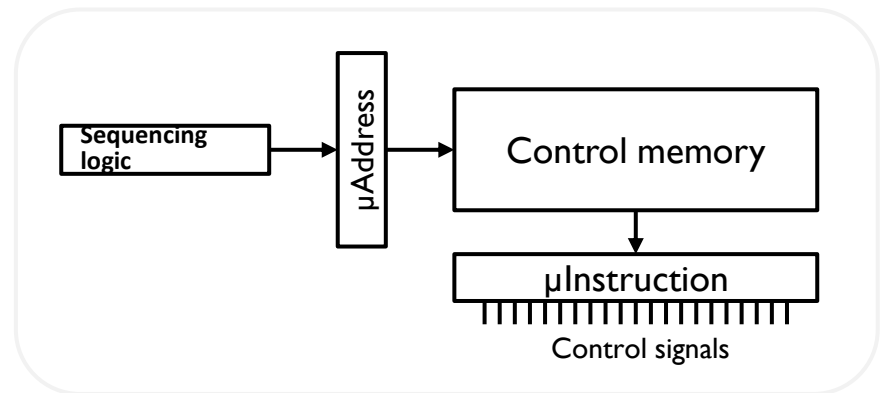
- ▶ Construction by means of logic gates, following logic design methods.
- ▶ Characteristics:
 - ▶ Laborious and costly circuit design and tuning.
 - ▶ Difficult to modify:
 - ▶ Complete redesign.
 - ▶ Very fast (used in RISC computers).



Control Unit: programmable logic

microprogramming

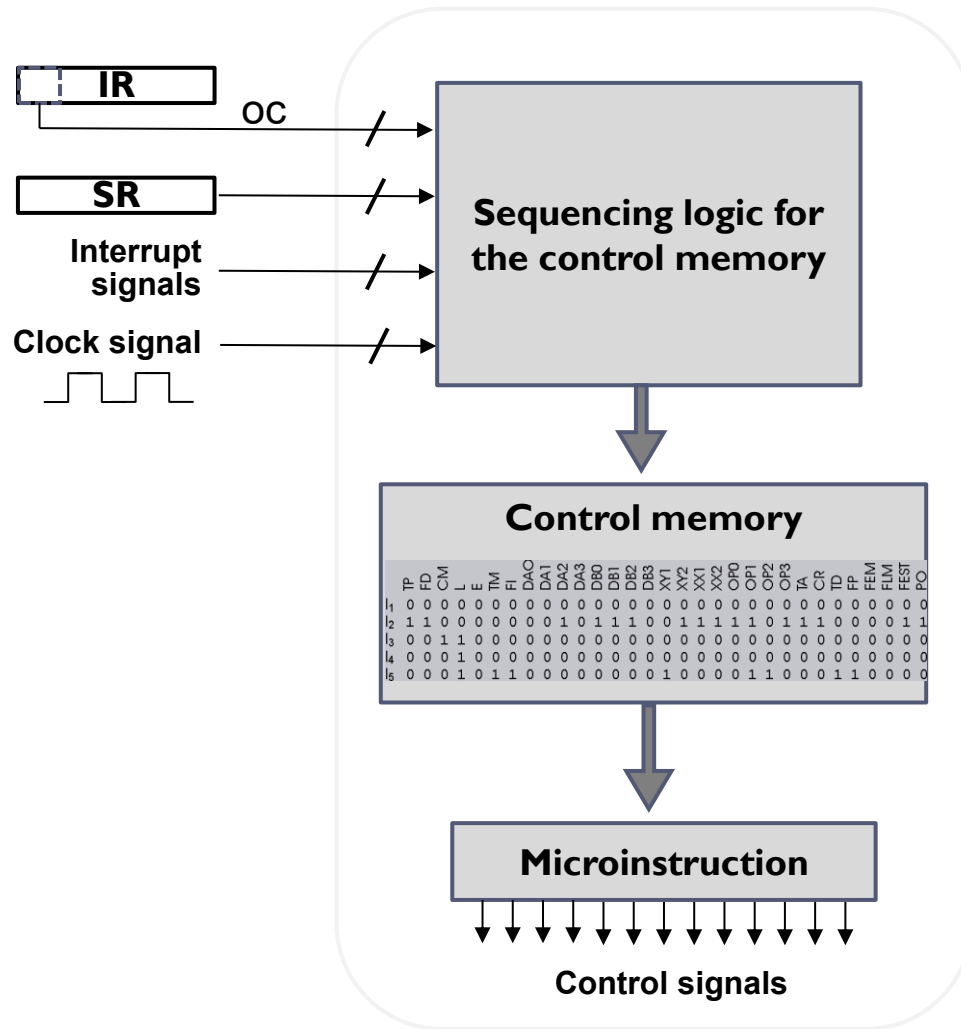
- ▶ Basic idea:
Use a memory (**control store**)
to store the signals of each cycle of each instruction..
- ▶ Characteristics:
 - ▶ Easy modification
 - ▶ Upgrade, expansion, etc.
 - ▶ E.g.: Certain consoles, routers, etc.
 - ▶ Easy to have complex instructions
 - ▶ E.g.: Diagnostic routines, etc.
 - ▶ Easy to have several sets of instructions
 - ▶ Other computers can be emulated.
 - ▶ Simple HW $\Rightarrow$ hard microcode



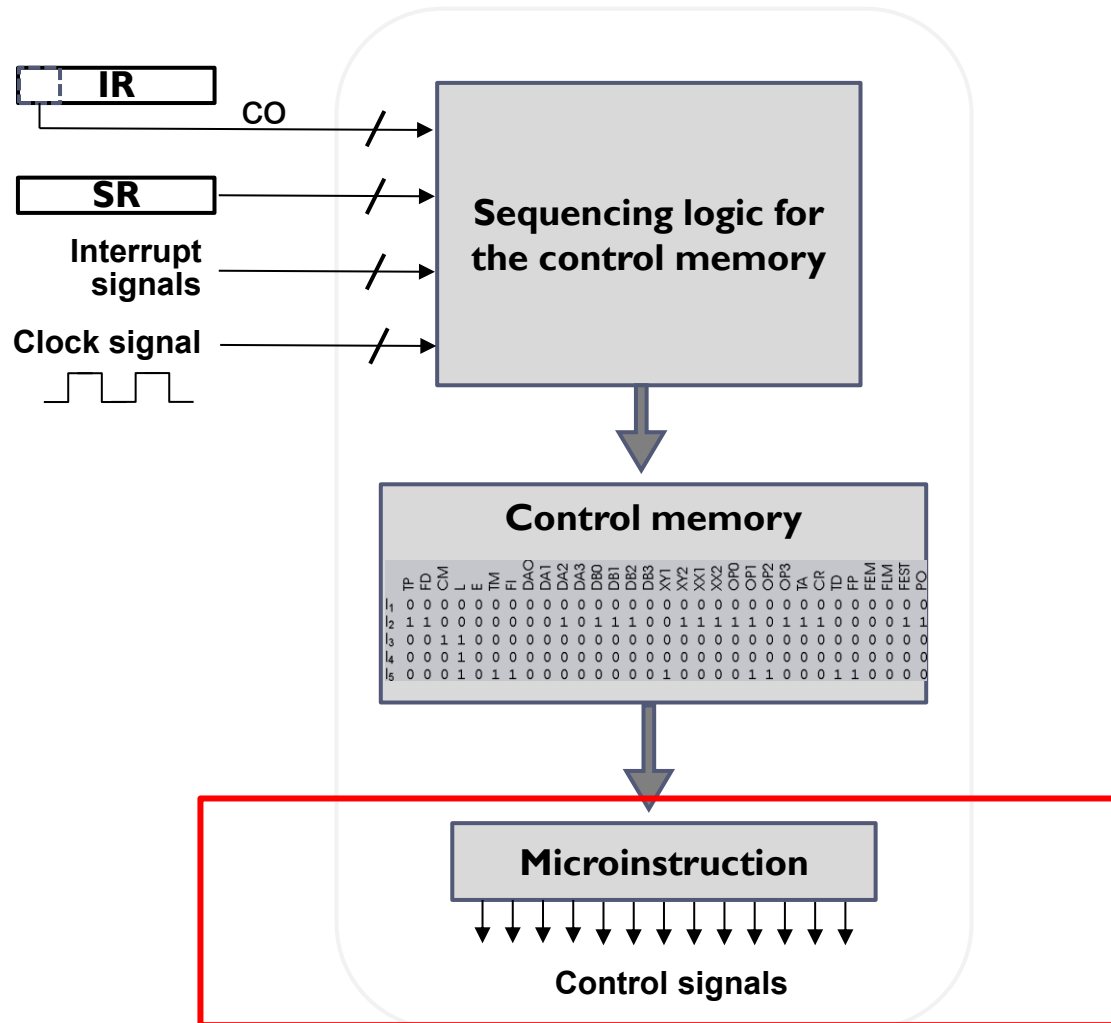
Contents

1. Computer elements
2. Processor organization
3. The control unit
4. Execution of instructions
5. Control unit design
 - a) Tasks in the design of a control unit
 - b) Microprogram control unit
 - c) Control unit in WepSIM
 - d) Example of a microprogrammed instruction set
6. Execution modes
7. Interrupts
8. Booting a computer
9. Performance and parallelism

General structure of a microprogrammed control unit



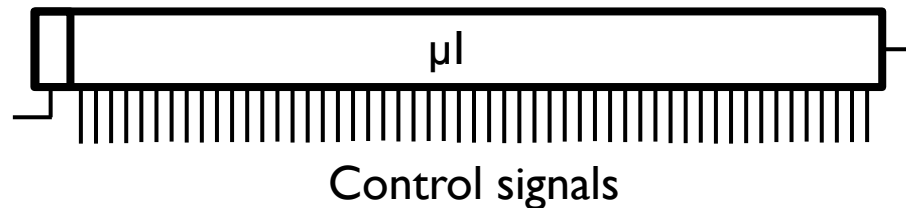
General structure of a microprogrammed control unit



Microinstruction format

- ▶ **Microinstruction format:**

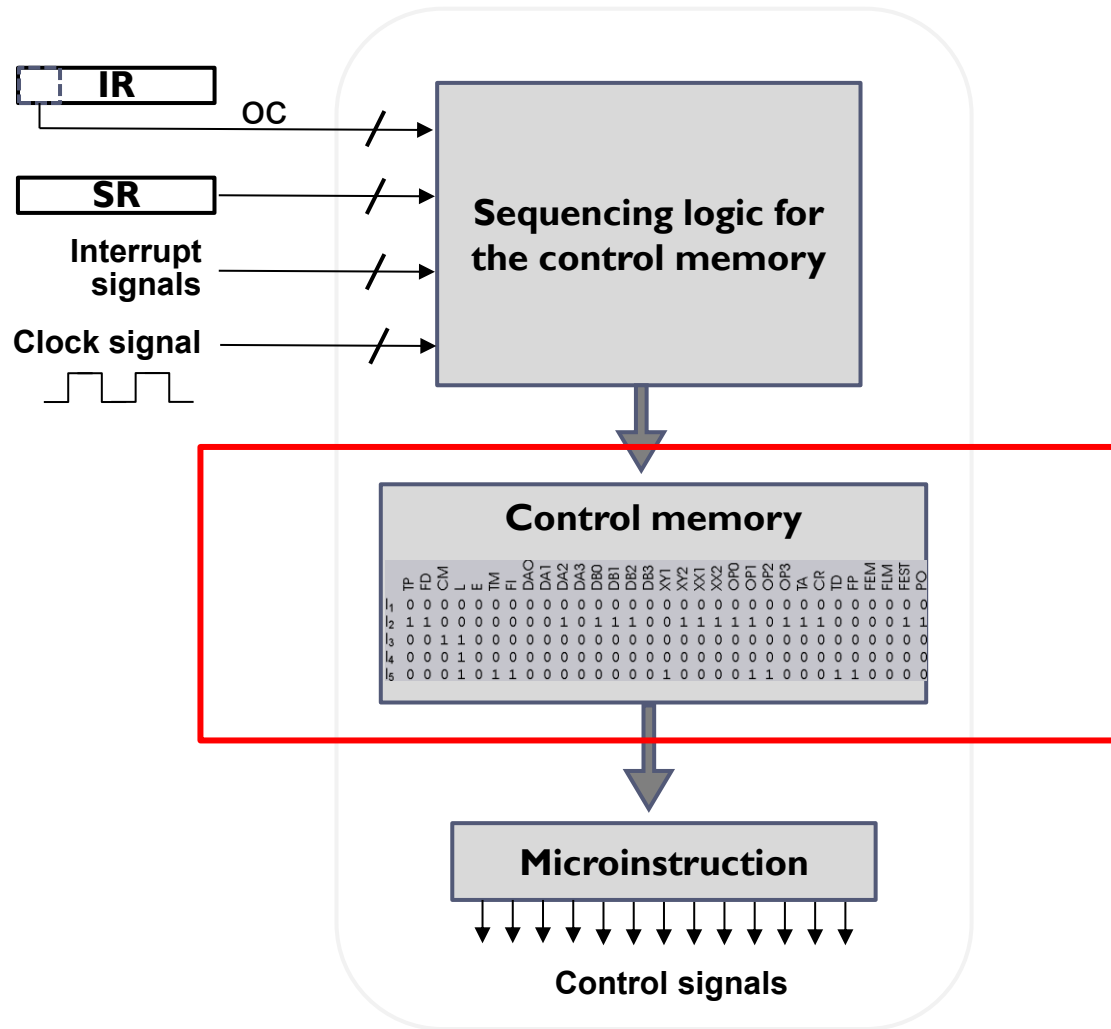
specifies the number of bits and the meaning of each bit.



- ▶ **Signals grouped into fields:**

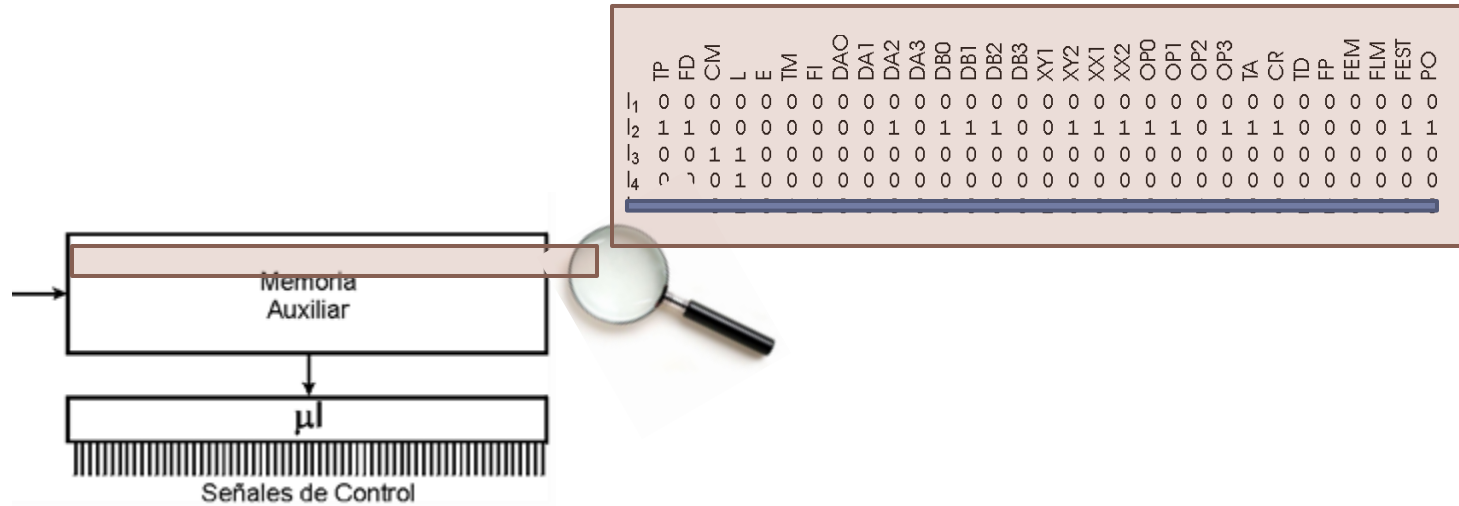
- ▶ Tristate bus signals
- ▶ ALU signals
- ▶ Registers file signals
- ▶ Main memory signals
- ▶ Multiplexor signals

General structure of a microprogrammed control unit



Microprogrammed control unit.

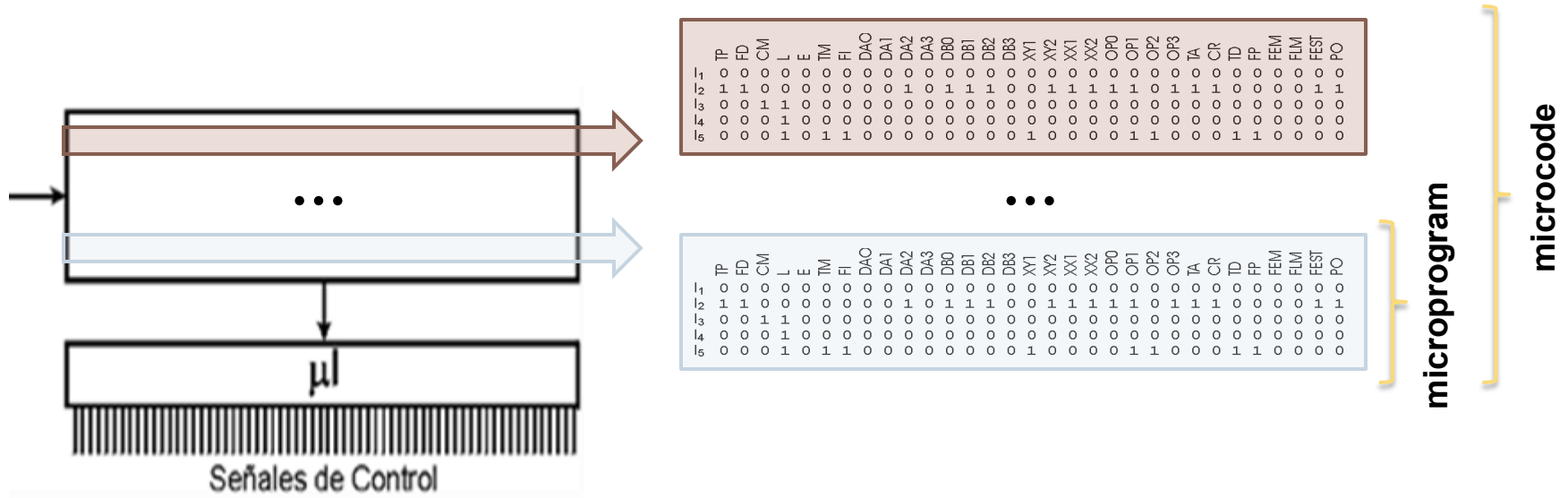
microinstructions



- ▶ **Microinstruction:** To each word defining the value of each control signal in a cycle of an instruction/fetch+IAC
- ▶ **The microinstructions...**
 - ▶ Are a list of 1's and 0's representing the state of each control signal during a period of one instruction.
 - ▶ Have one bit for each control signal.

Microprogrammed control unit.

microprogram and microcode

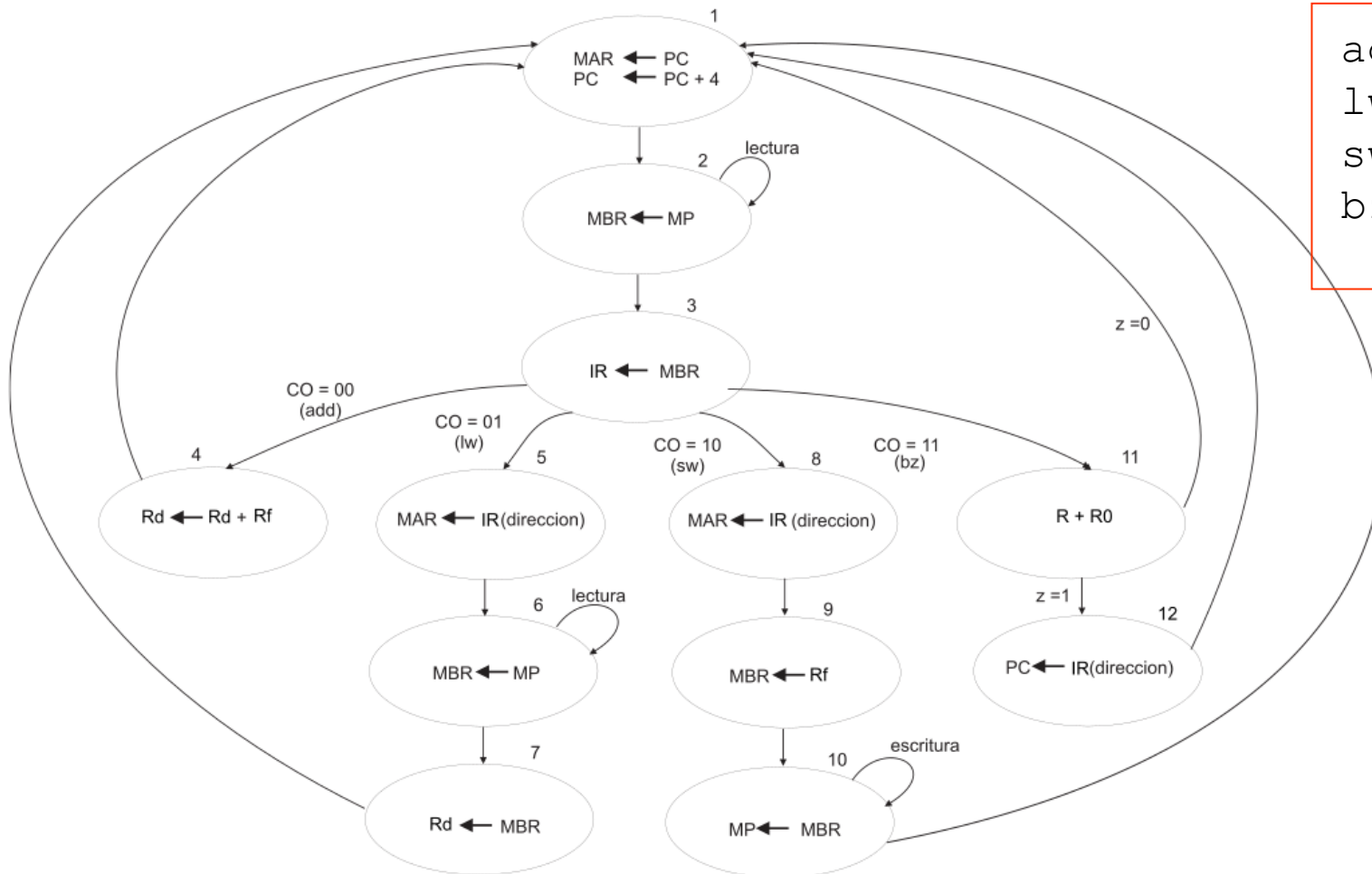


- ▶ **microprogram**: ordered list of microinstructions, which represent the chronogram of a machine instruction.
- ▶ **microcode**: set of microprograms of a machine.

Example: State machine

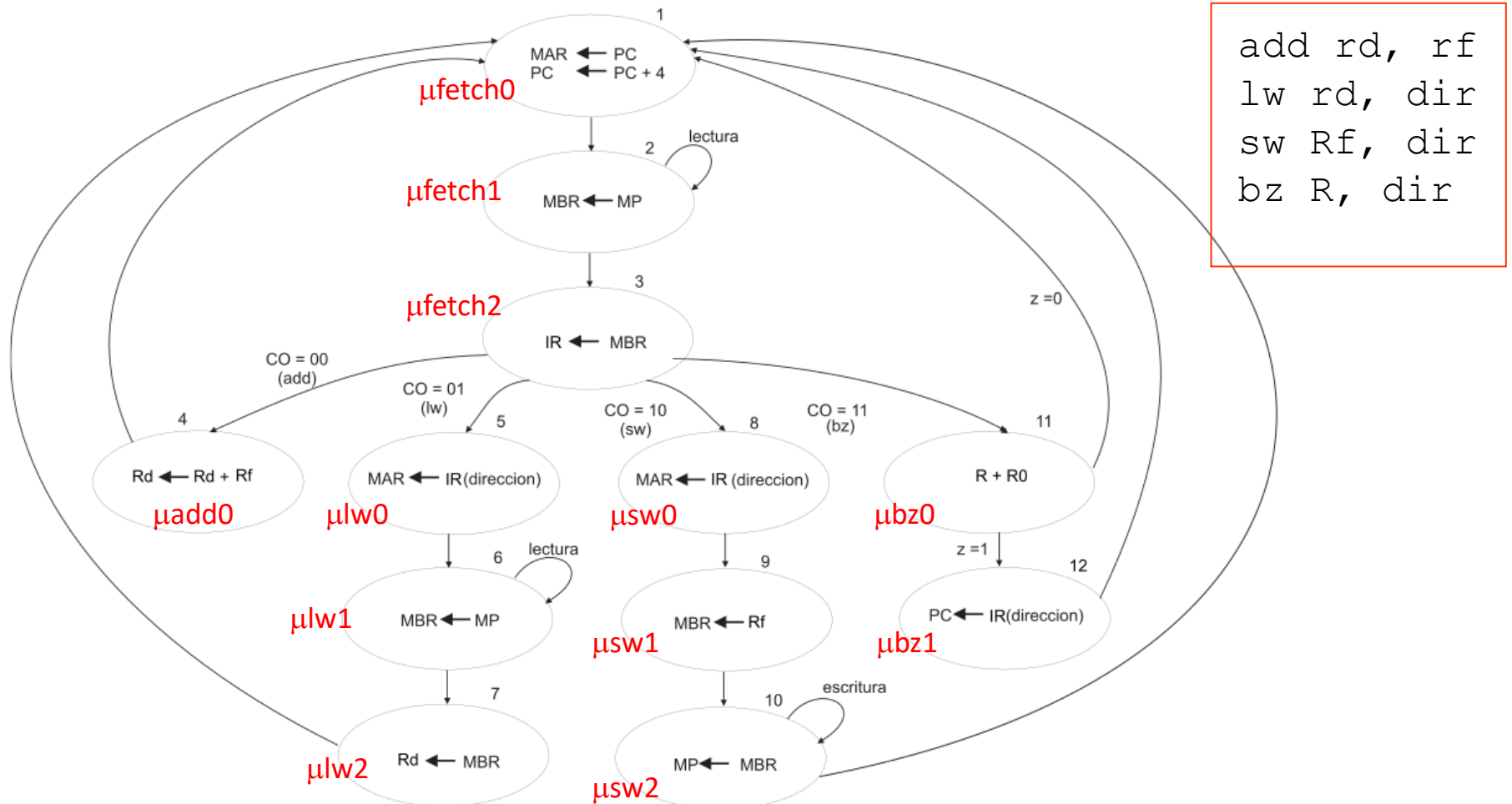
Example for a computer with only 4 machine instructions

```
add rd, rf
lw rd, dir
sw Rf, dir
bz R, dir
```



Example: associated microinstructions

Example for a computer with only 4 machine instructions



Example: microcode

```
add r1, r2
lw r1, dir
bz dir
sw r1
```

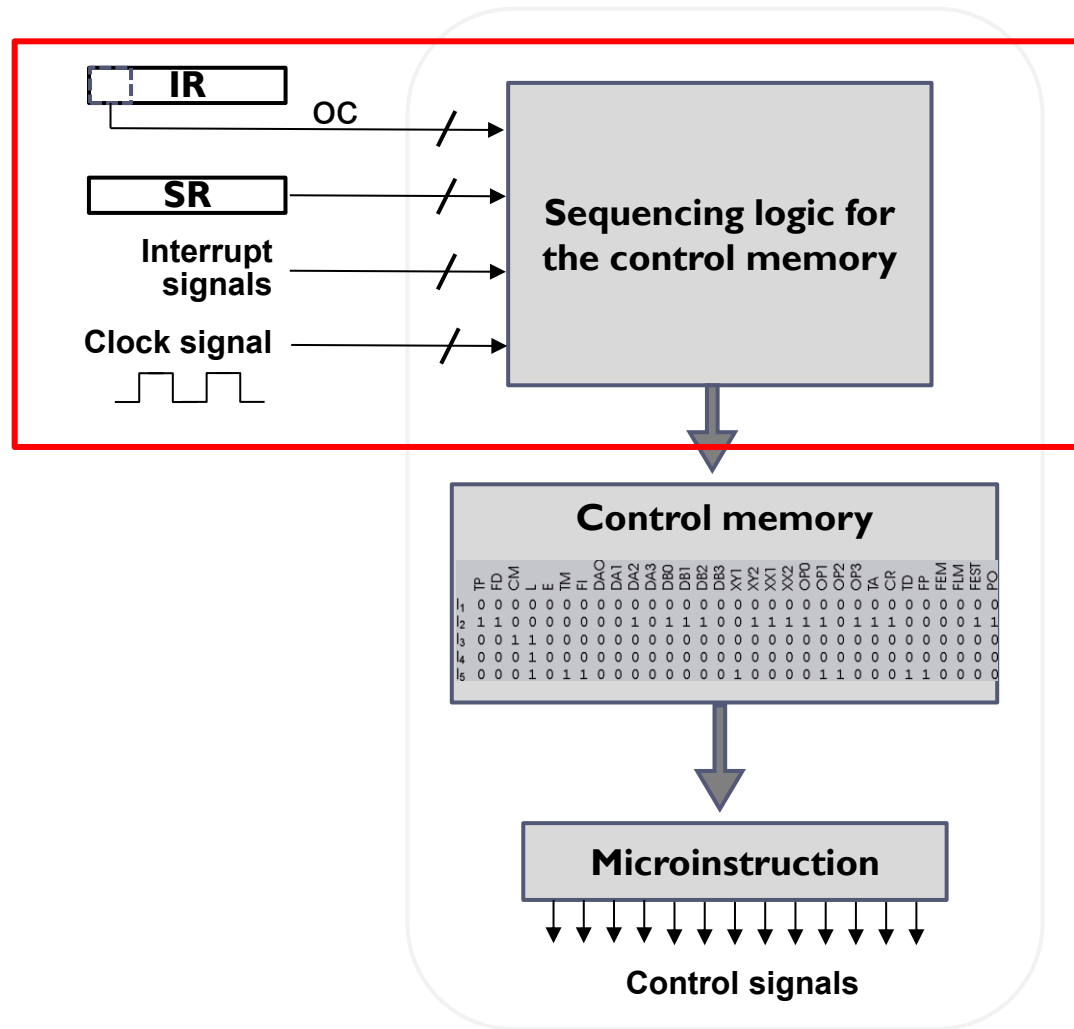
	C0	C1	C2	C3	C4	C5	C6	C7	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	LE	MA	MB1	MB0	M1	M2	M7	R	W	Ta	Td	
μfetch0	1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	fetch
μfetch1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	
μfetch2	0	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
μadd0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	add
μlw0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
μlw1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	lw
μlw2	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	
μsw0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
μsw1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	sw
μsw3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	
μbz0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	bz
μbz1	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Contents of the control memory



- ▶ **FETCH: get next instruction**
 - ▶ IAC: interrupt acknowledge cycle.
 - ▶ $IR \leftarrow \text{Mem}[\text{PC}], \text{PC}++$, jump-to-O.C.
- ▶ **Microprograms:**
one for every machine instruction
 - ▶ fetch rest of operands (if any)
 - ▶ Updates PC on multi-word instructions
 - ▶ Execute the instruction
 - ▶ Jump to FETCH

General structure of a microprogrammed control unit



Microprogrammed control unit structure

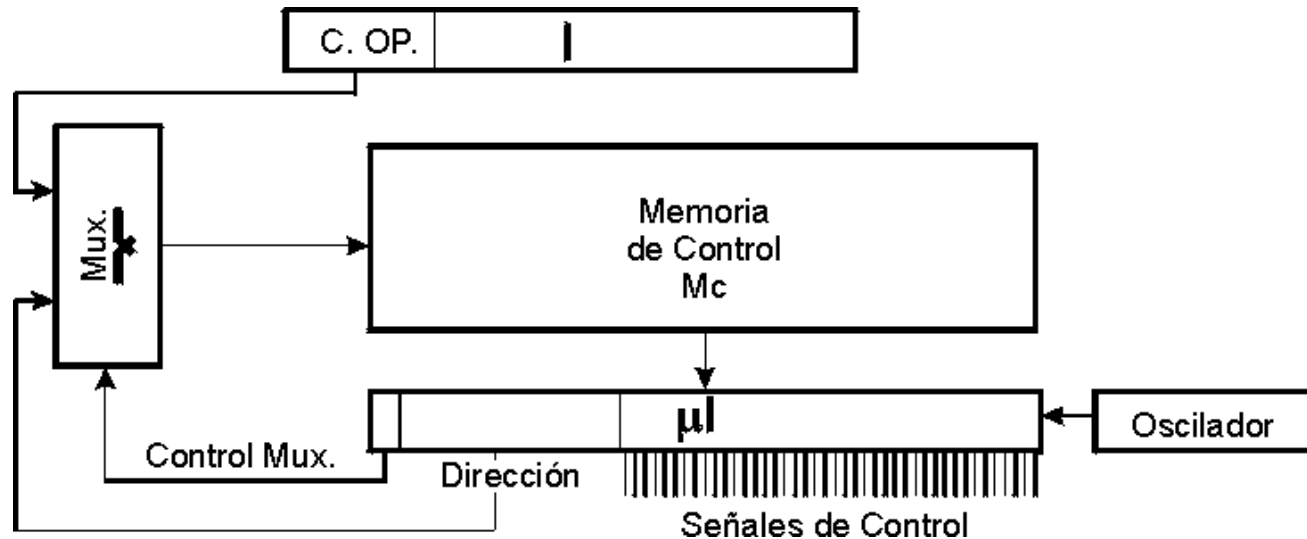
▶ Three basic conditions:

1. Sufficient control memory to store all microprograms corresponding to all instructions.
2. Procedure for associating each instruction with its microprogram
 - ▶ Procedure that converts the instruction operation code to the control memory address where your microprogram starts..
3. Sequencing mechanism to read successive microinstructions, and to branch to another microprogram when the current one is finished.

▶ Two alternatives:

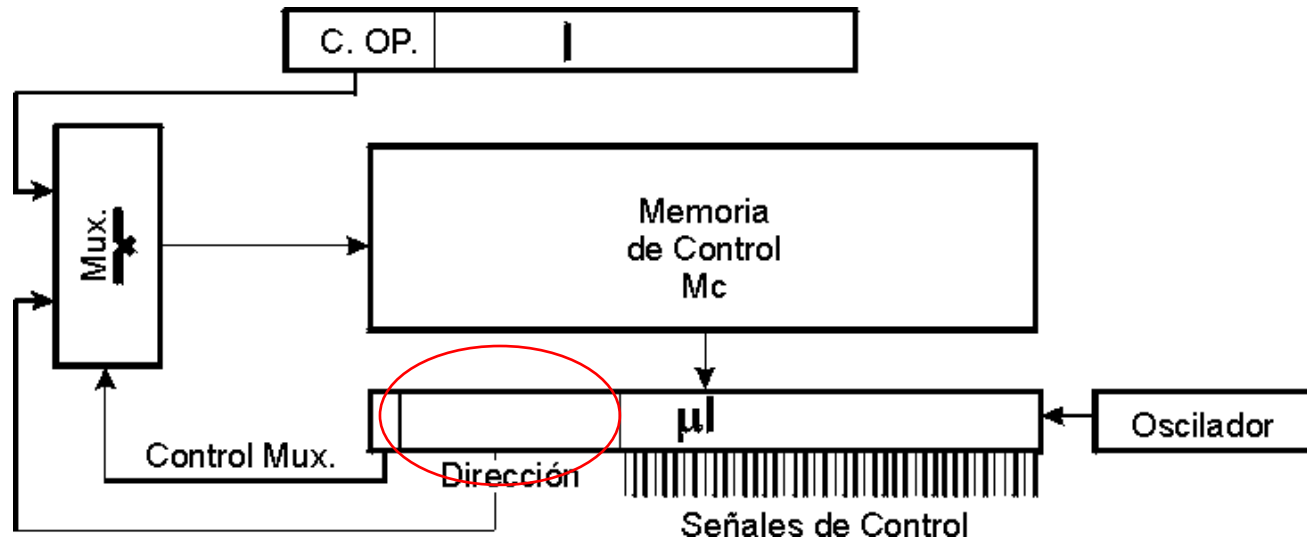
1. Explicit sequencing.
2. Implicit sequencing.

Microprogrammed C.U. structure with **explicit** sequencing



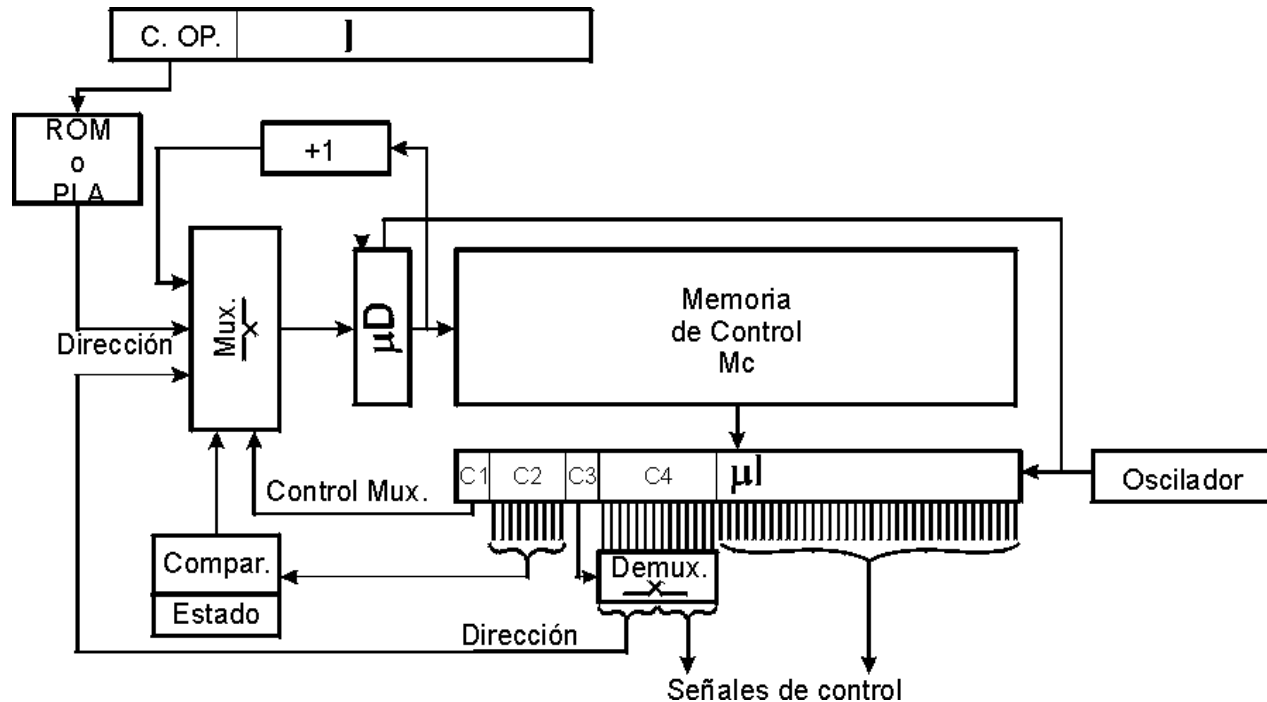
- ▶ Control memory stores all μ programs, where each μ instruction provides the next μ instruction μ address
- ▶ The OC represents the μ Address of the first μ instruction associated with the machine instruction.

Microprogrammed C.U. structure with **explicit** sequencing



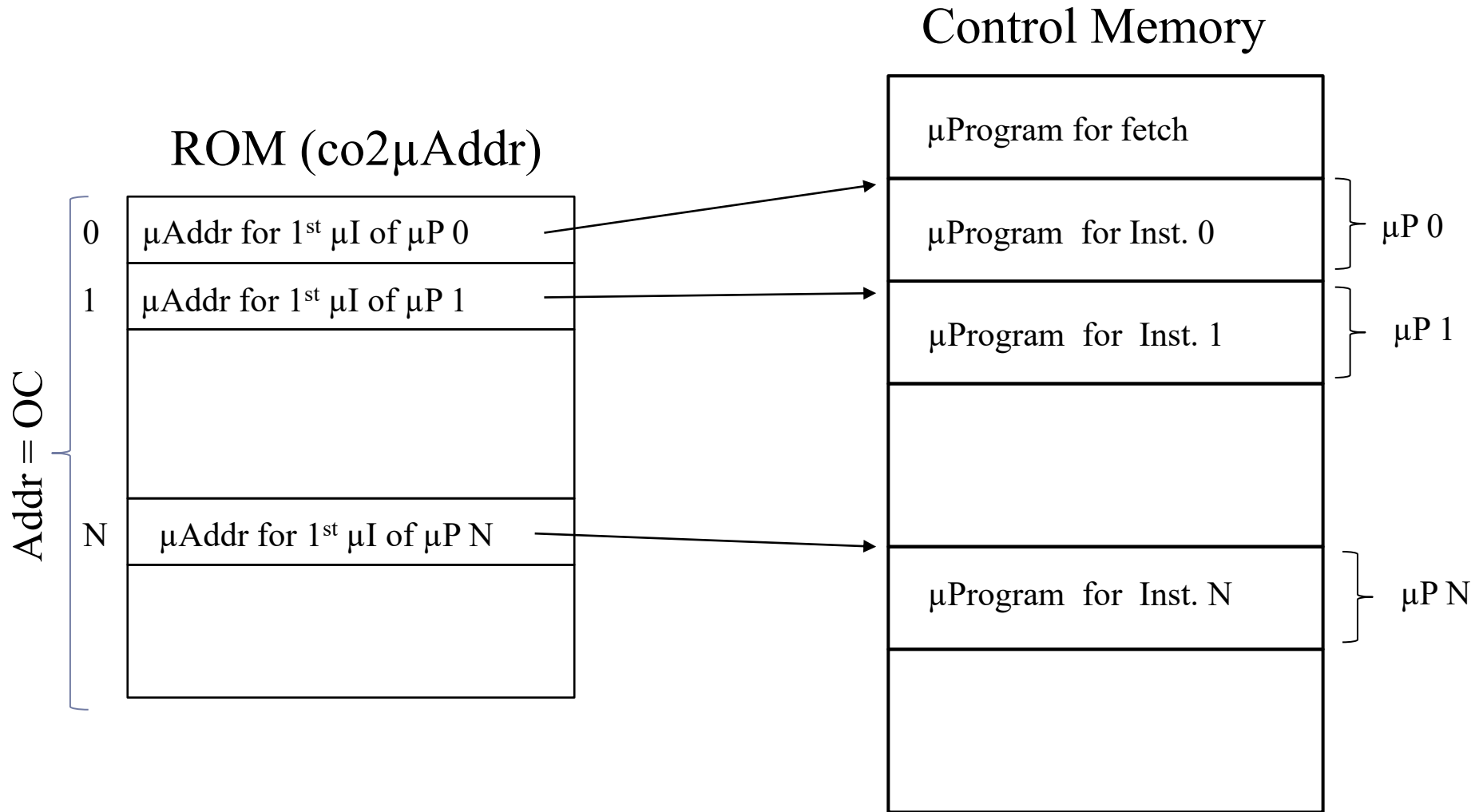
- ▶ Control memory stores all μ programs, where each μ instruction provides the next μ instruction μ address
- ▶ **Problem:** large amount of control memory for instruction sequencing, required stores the next μ address

Microprogrammed C.U. structure with **implicit** sequencing

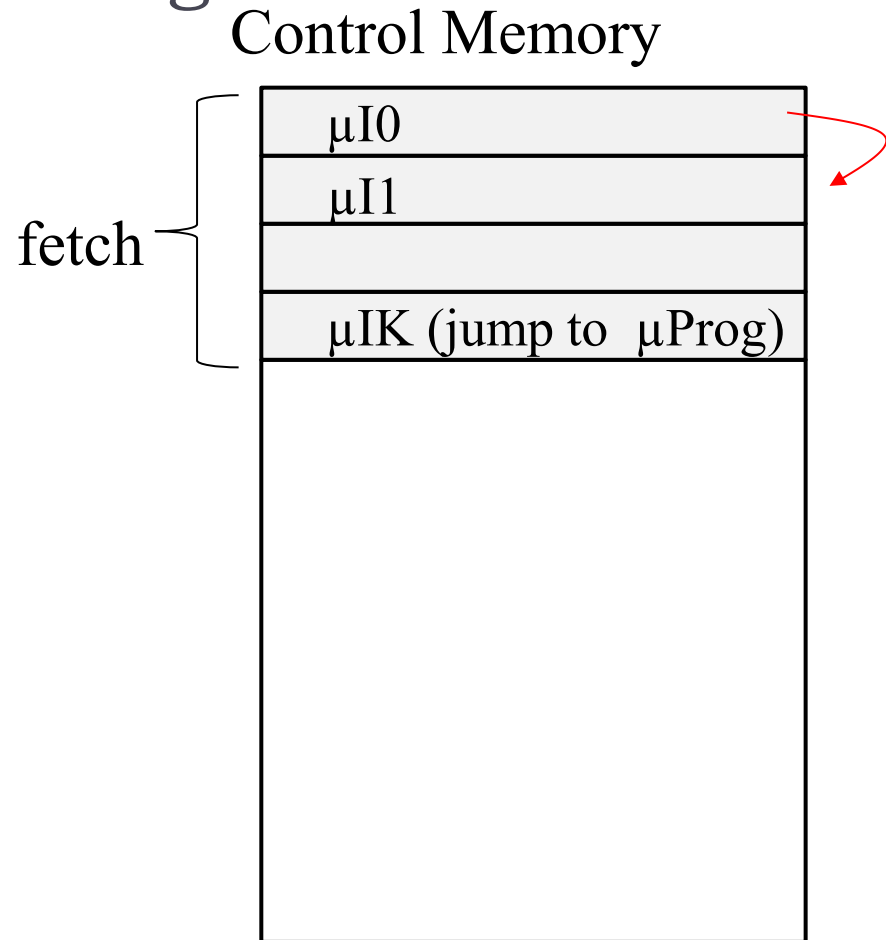
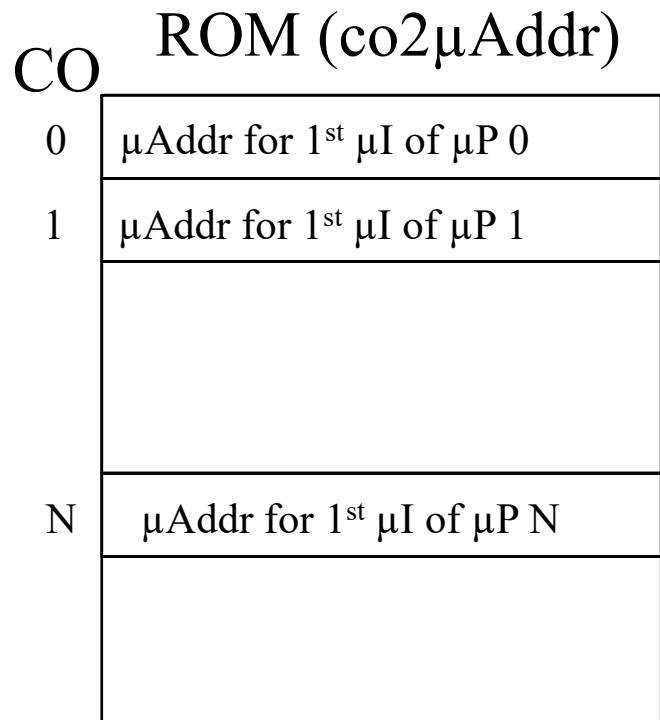


- ▶ Control memory stores all microprograms consecutively in the control memory.
- ▶ The ROM/PLA associates each instruction with its microprogram (first μ address, μ conditional μ instruction (+1), μ conditional μ bifurcations or μ loops).
- ▶ Next μ instruction (+1), conditional μ bifurcations or μ loops

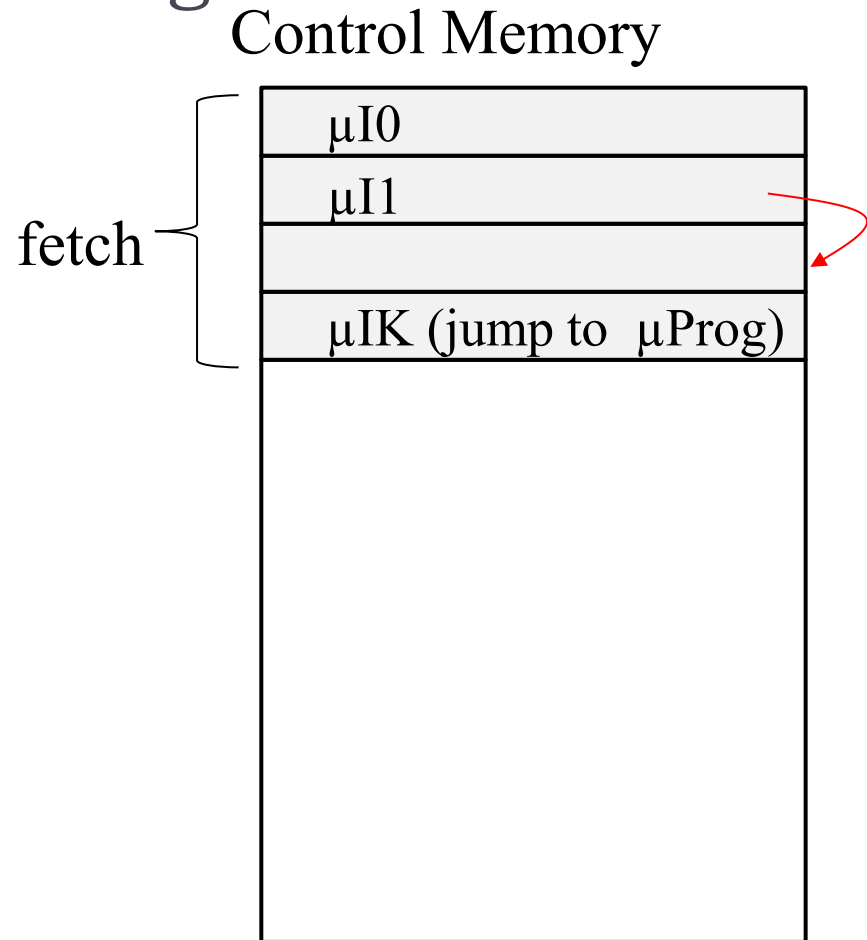
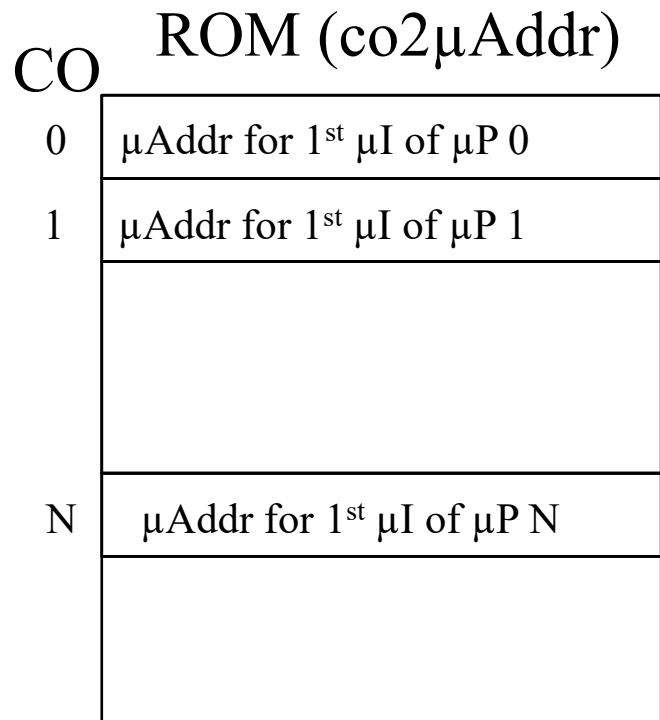
Example of Control Unit with **implicit** sequencing



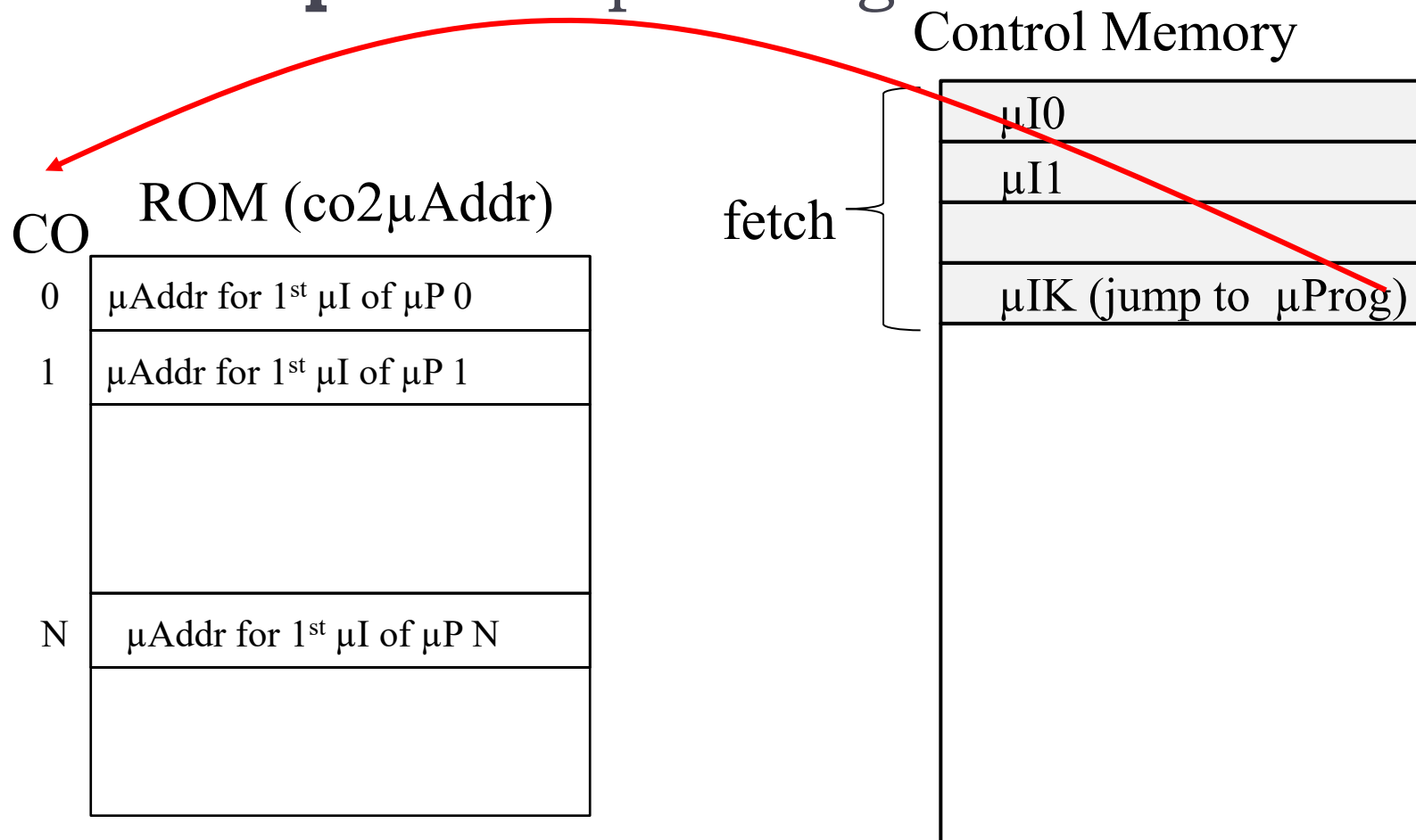
Example of Control Unit with **implicit** sequencing



Example of Control Unit with **implicit** sequencing

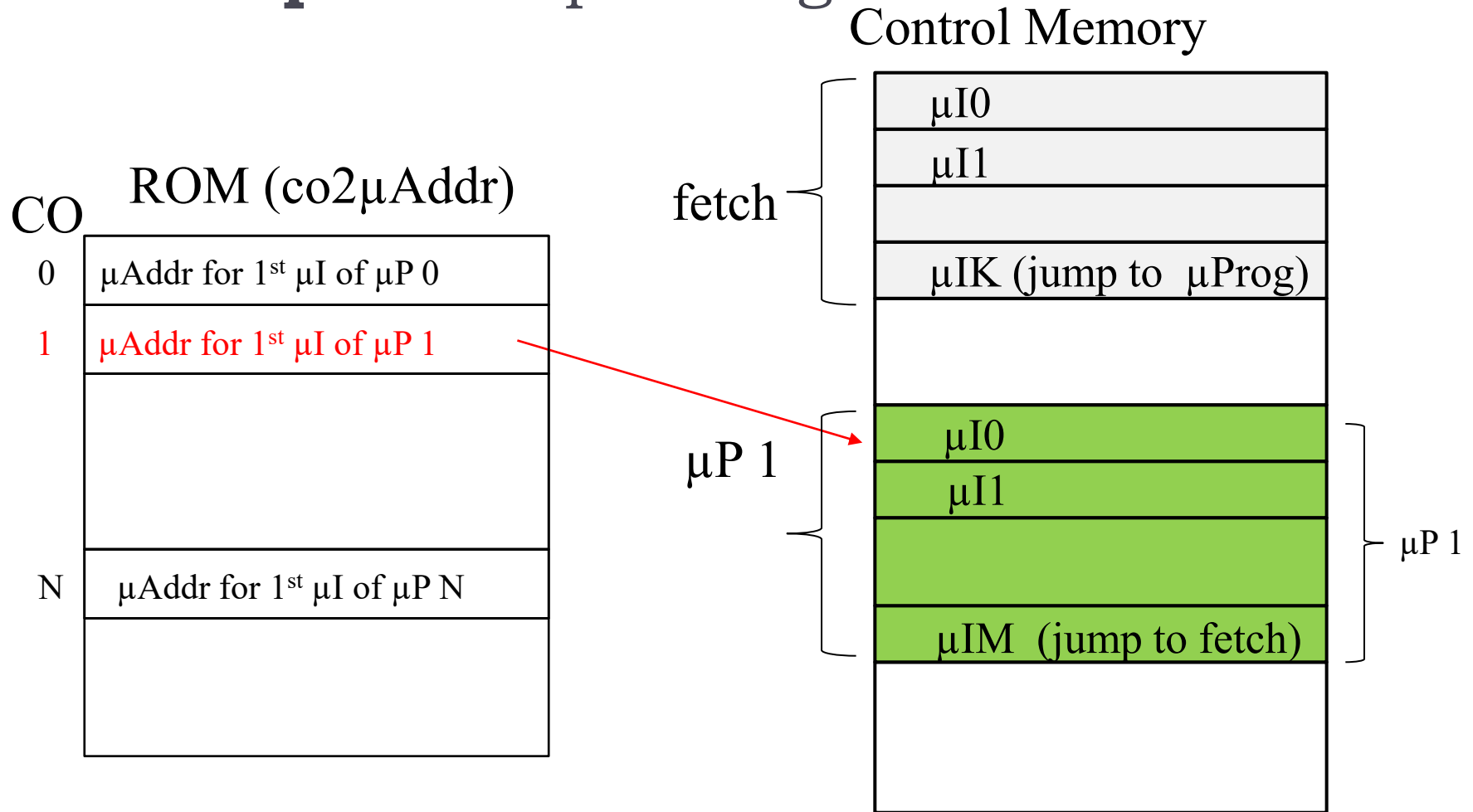


Example of Control Unit with **implicit** sequencing



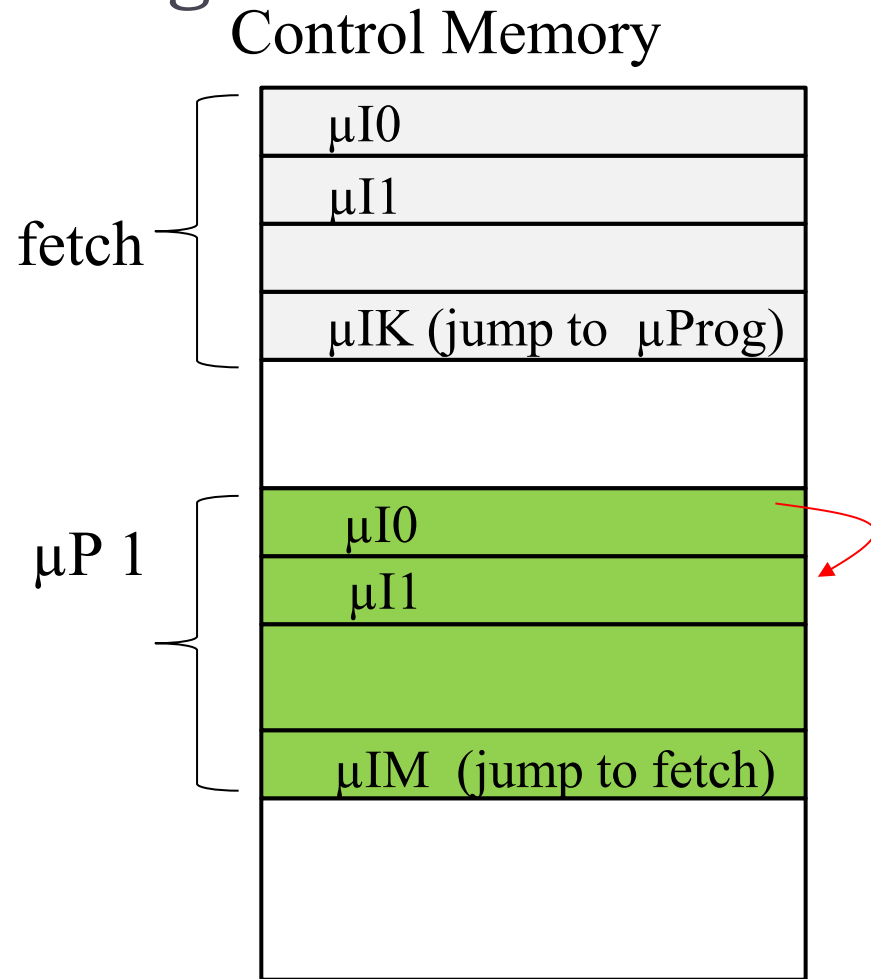
The Operation Code (OC) is at the Instruction Register (IR)

Example of Control Unit with **implicit** sequencing



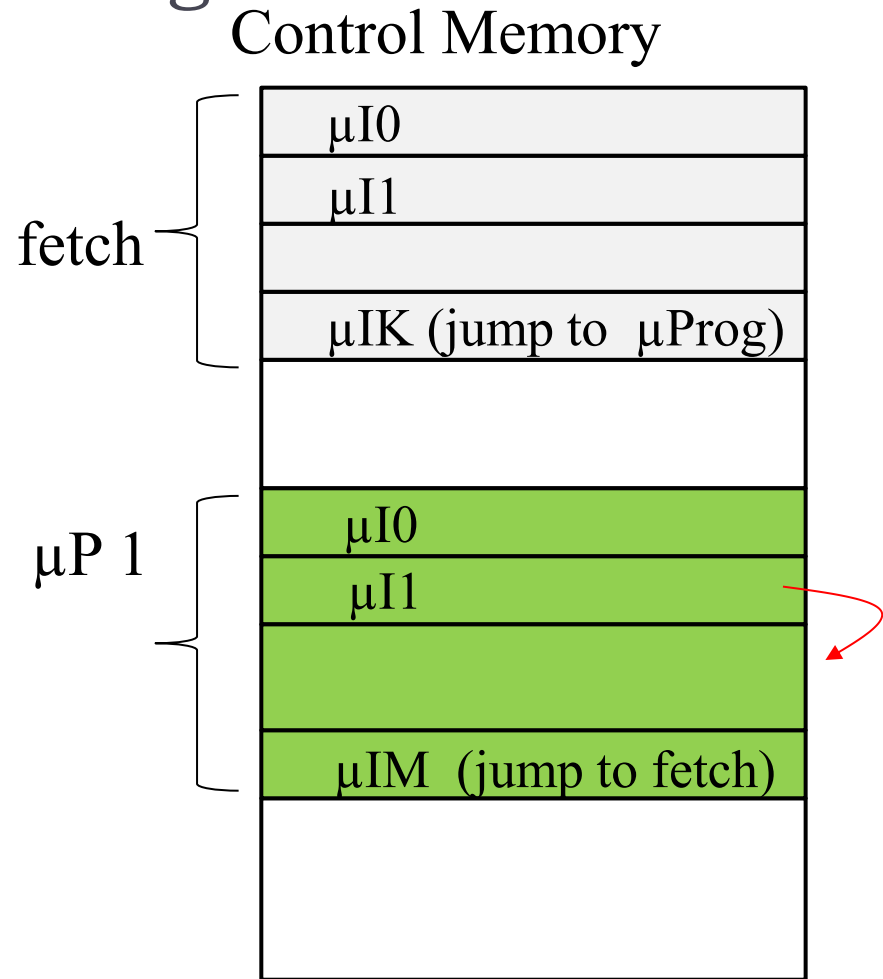
Example of Control Unit with **implicit** sequencing

CO	ROM (co2 μ Addr)
0	μ Addr for 1 st μ I of μ P 0
1	μ Addr for 1 st μ I of μ P 1
N	μ Addr for 1 st μ I of μ P N



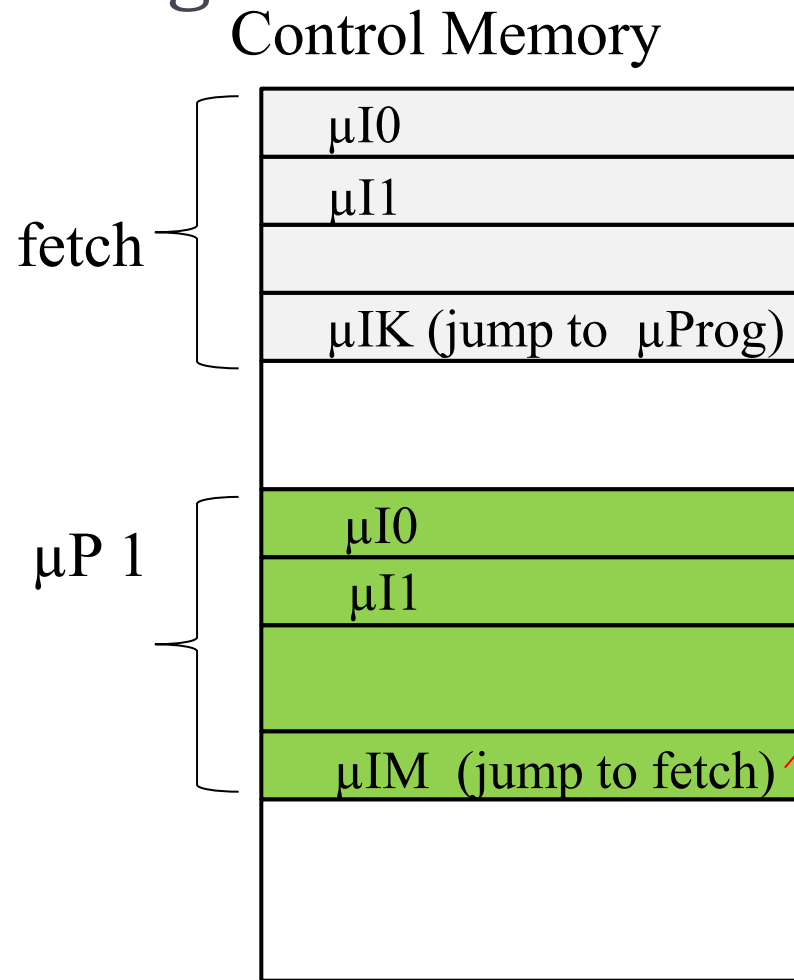
Example of Control Unit with **implicit** sequencing

CO	ROM (co2 μ Addr)
0	μ Addr for 1 st μ I of μ P 0
1	μ Addr for 1 st μ I of μ P 1
N	μ Addr for 1 st μ I of μ P N



Example of Control Unit with **implicit** sequencing

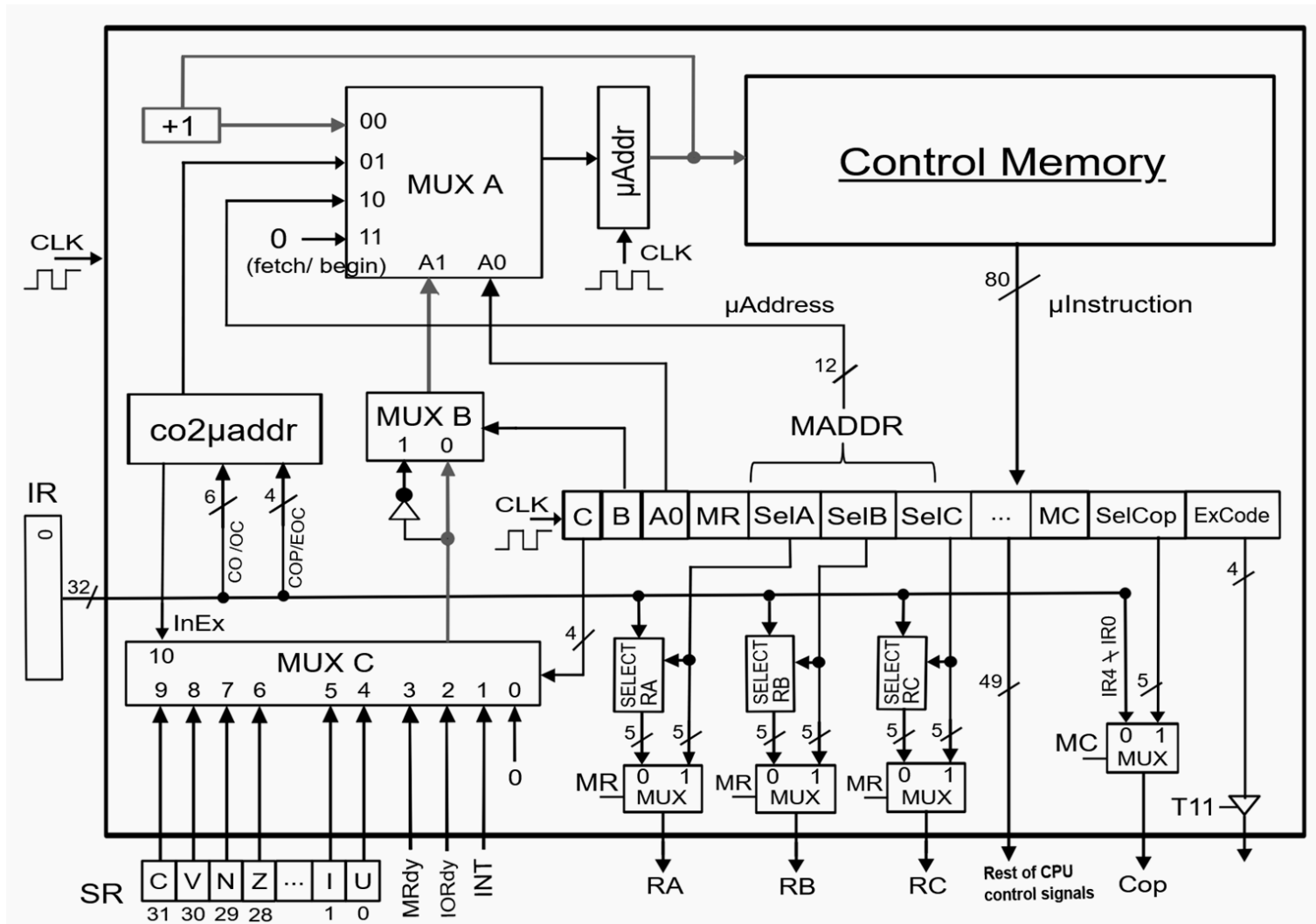
CO	ROM (co2 μ Addr)
0	μ Addr for 1 st μ I of μ P 0
1	μ Addr for 1 st μ I of μ P 1
N	μ Addr for 1 st μ I of μ P N



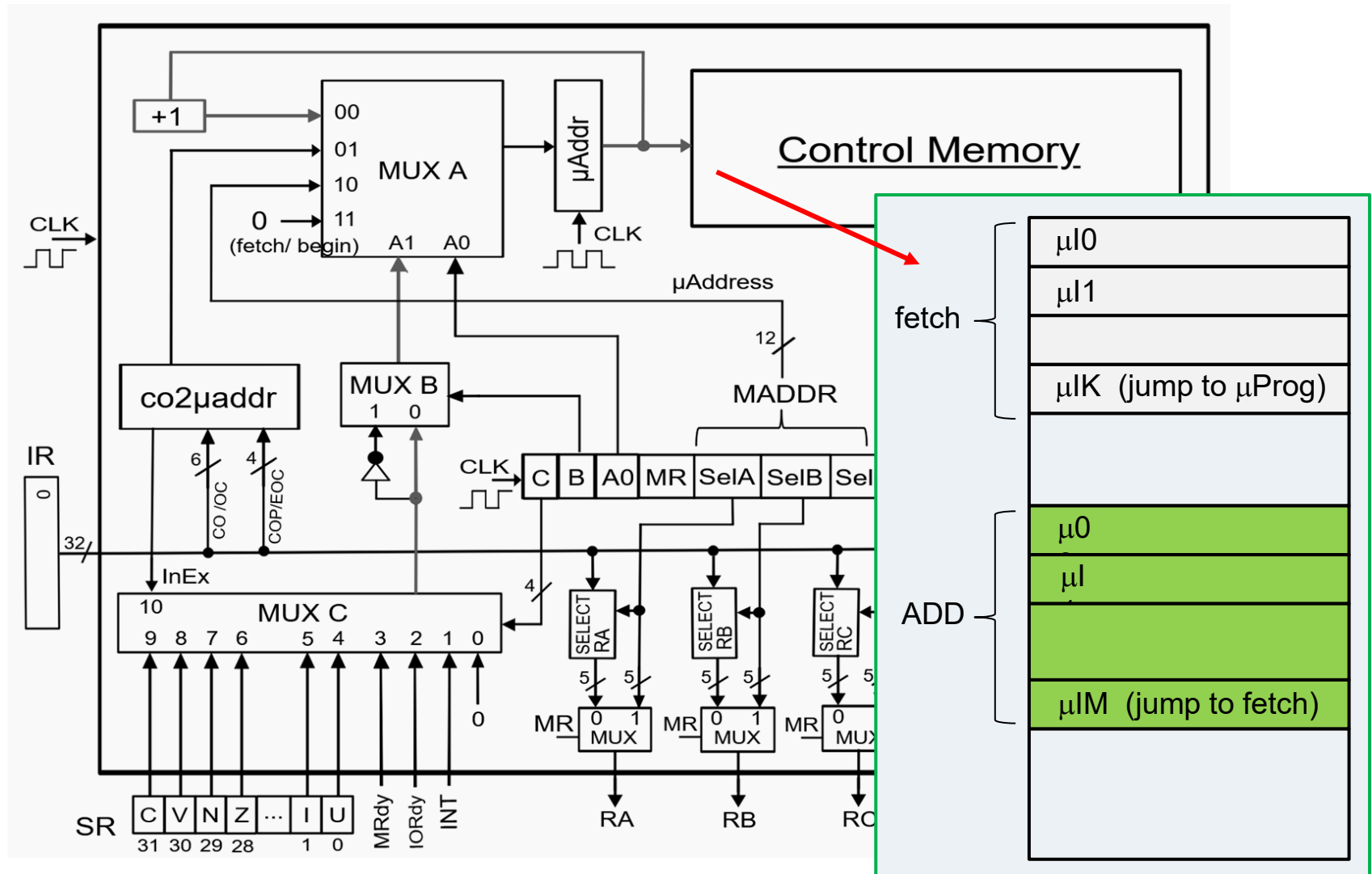
Contents

1. Computer elements
2. Processor organization
3. The control unit
4. Execution of instructions
5. **Control unit design**
 - a) Tasks in the design of a control unit
 - b) Microprogram control unit
 - c) **Control unit in WepSIM**
 - d) Example of a microprogrammed instruction set
6. Execution modes
7. Interrupts
8. Booting a computer
9. Performance and parallelism

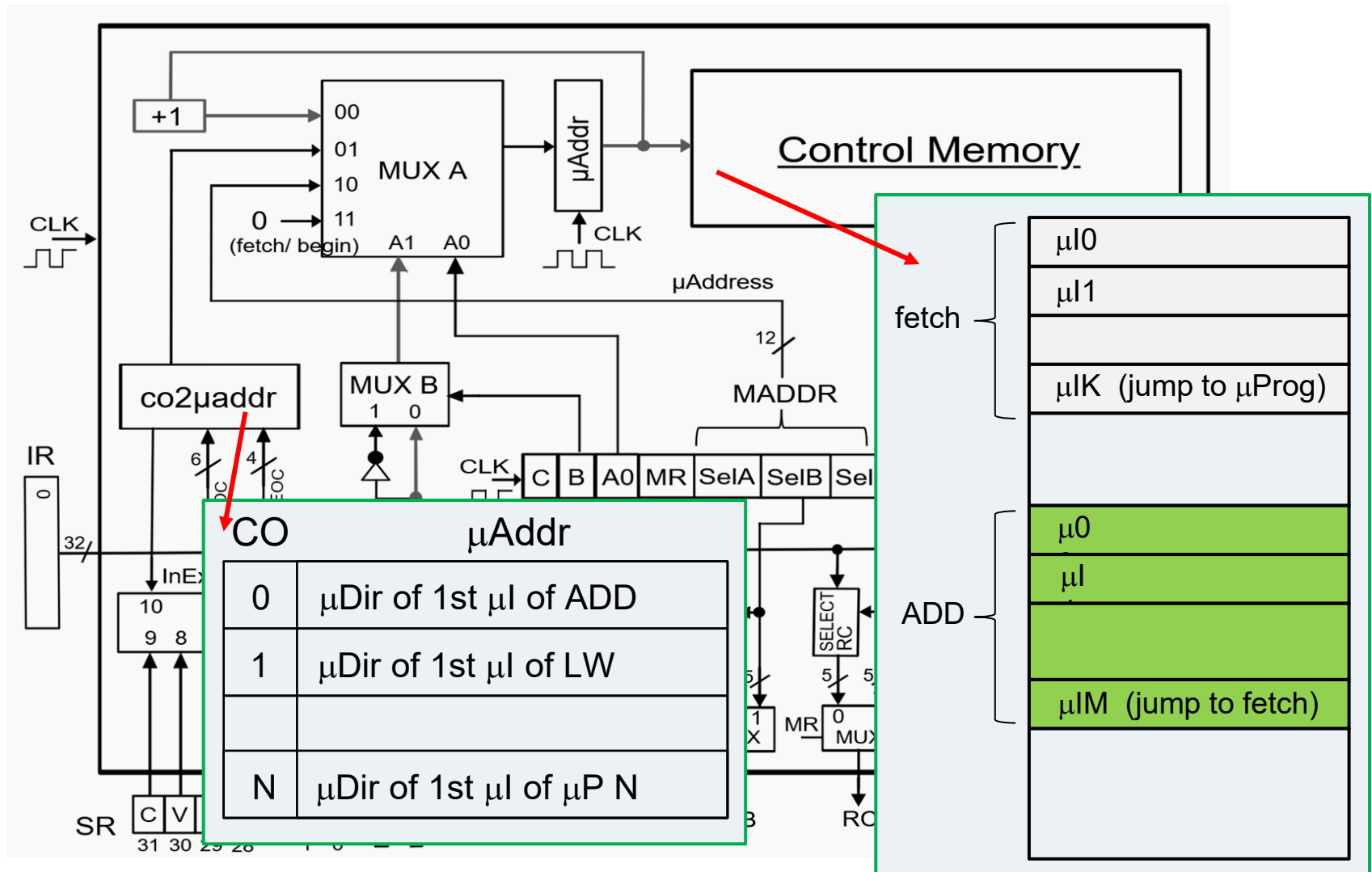
Control Unit in WepSIM



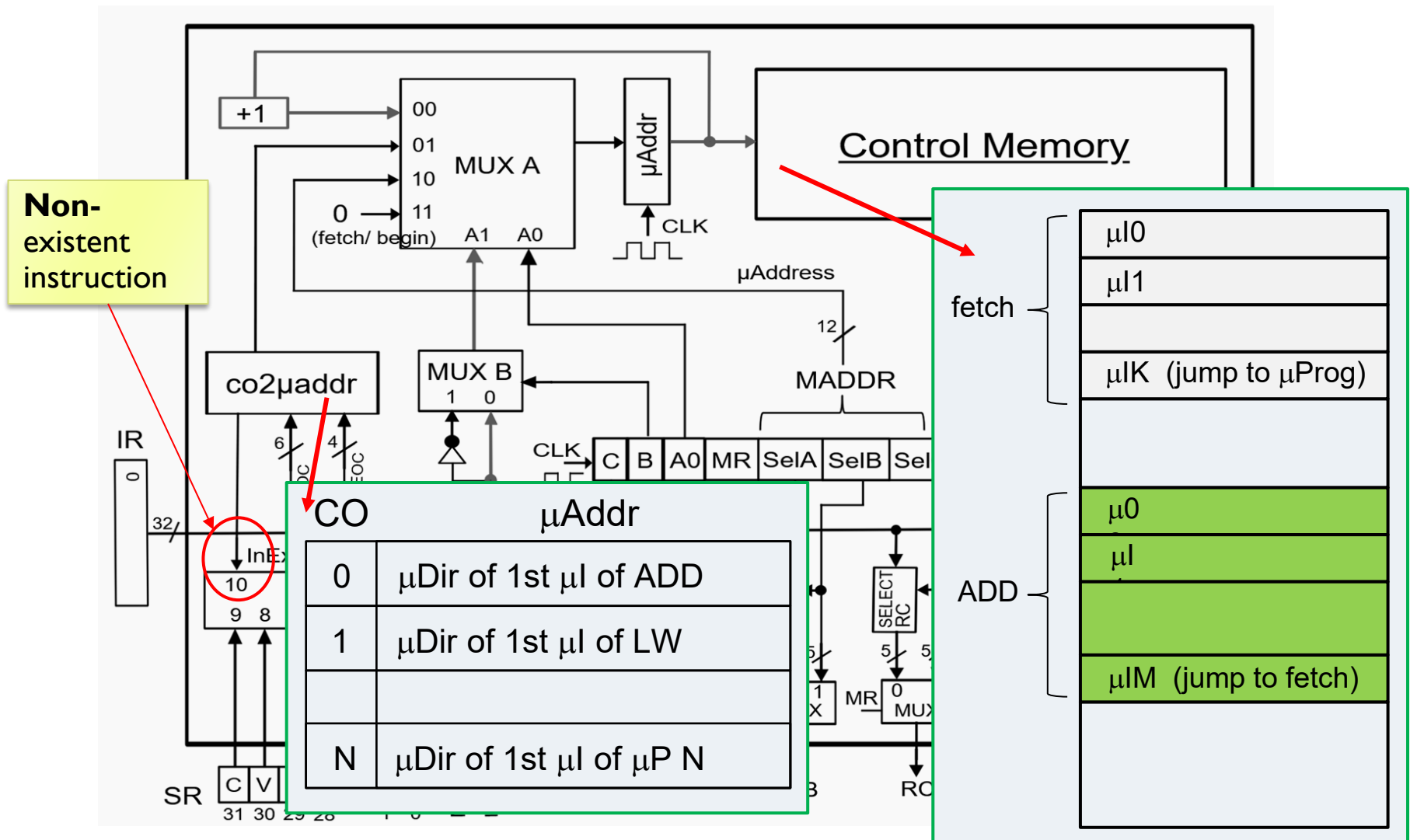
Control Unit in WepSIM



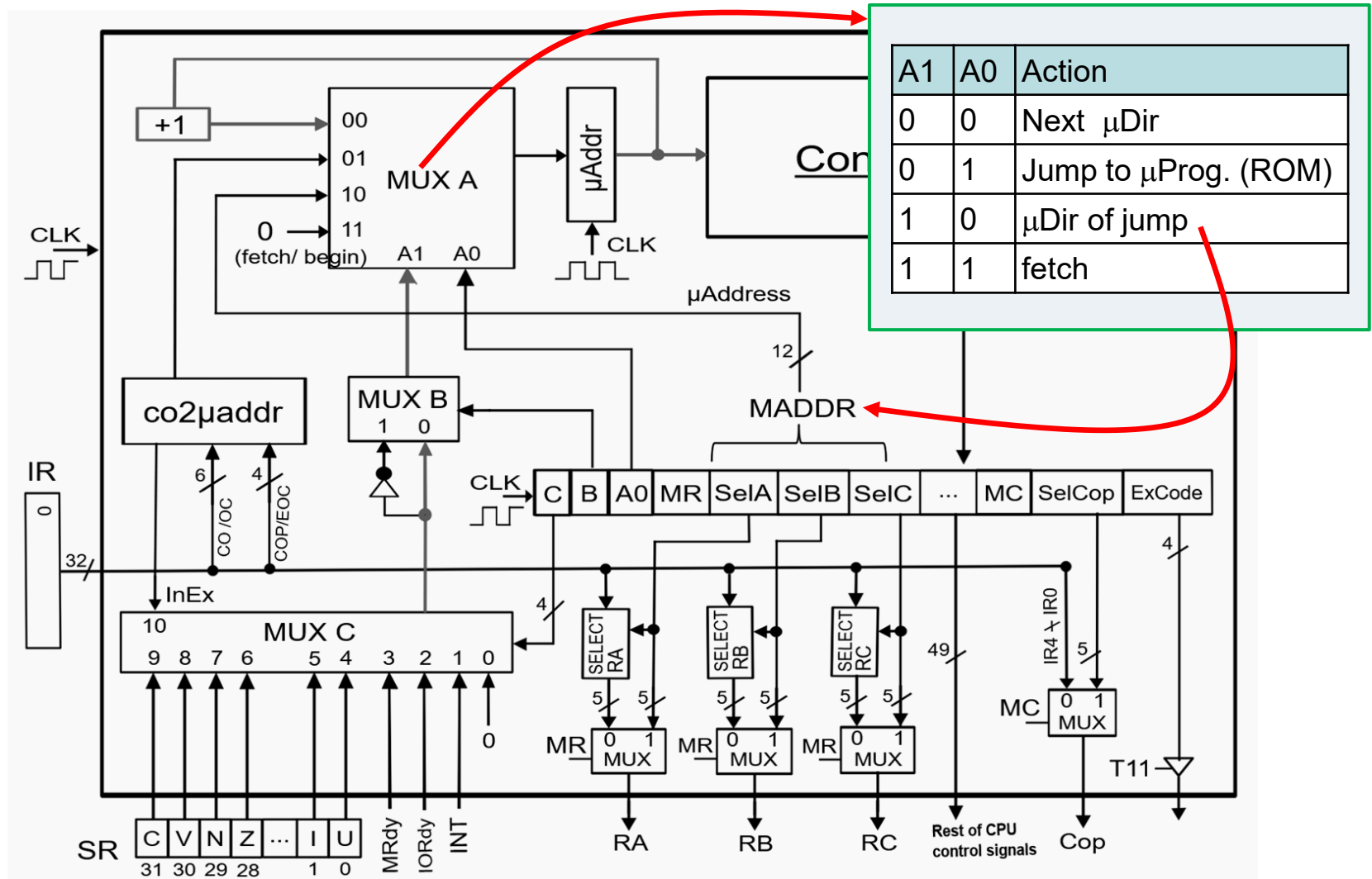
Control Unit in WepSIM



Control Unit in WepSIM



Control Unit in WepSIM



Examples of more frequent jumps

Elemental operations with CU

- ▶ **Jump to address 000100011100 (12 bits) if Z = 1. Otherwise jump to the next one.**

Elemental operation	Signals
If (Z) $\mu\text{PC}=000100011100$	$A0=0, B=0, C=0110_2, \text{mADDR}=000100011100_2$

- ▶ **Salto incondicional a la dirección 000100011111**

Elemental operation	Signals
$\mu\text{PC}=000100011111$	$A0=0, B=1, C=0000_2, \text{mADDR}=000100011111_2$

- ▶ **Jump to first μ address of the μ program related to OC**

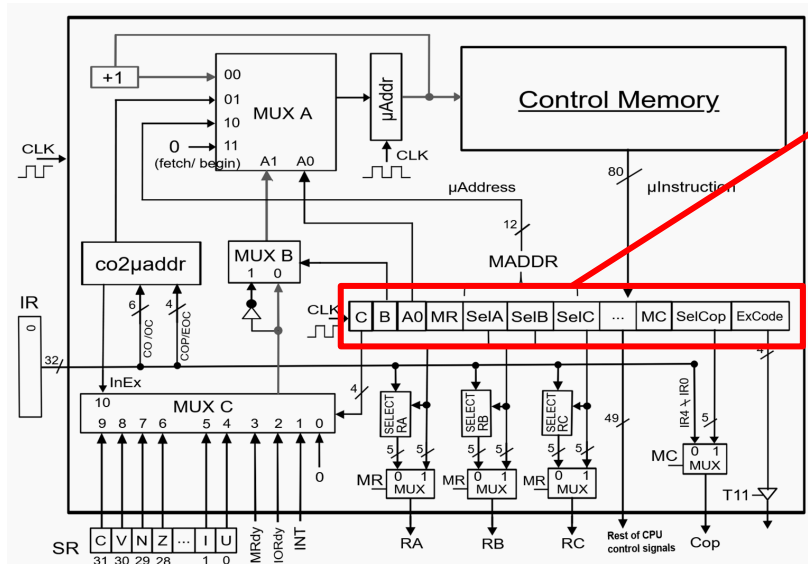
Elemental operation	Signals
Jump to OC	$A0=1, B=0, C=0000_2$

Control Unit in WepSIM

A0	B	C3	C2	C1	C0	Acción
0	0	0	0	0	0	Siguiente μ Dirección
0	1	0	0	0	0	Salto incondicional a MADDR
0	0	0	0	0	1	Salto condicional a MADDR si INT = 1 (*)
0	1	0	0	1	0	Salto condicional a MADDR si IORdy = 0 (*)
0	1	0	0	1	1	Salto condicional a MADDR si MRdy = 0 (*)
0	0	0	1	0	0	Salto condicional a MADDR si U = 1 (*)
0	0	0	1	0	1	Salto condicional a MADDR si I = 1 (*)
0	0	0	1	1	0	Salto condicional a MADDR si Z = 1 (*)
0	0	0	1	1	1	Salto condicional a MADDR si N = 1 (*)
0	0	1	0	0	0	Salto condicional a MADDR si O = 1 (*)
1	0	0	0	0	0	Salto a μ Prog. (ROM c02 μ addr)
1	1	0	0	0	0	Salto a fetch (μ Dir = 0)

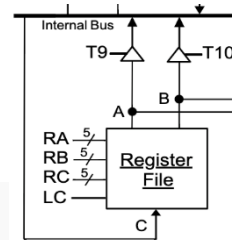
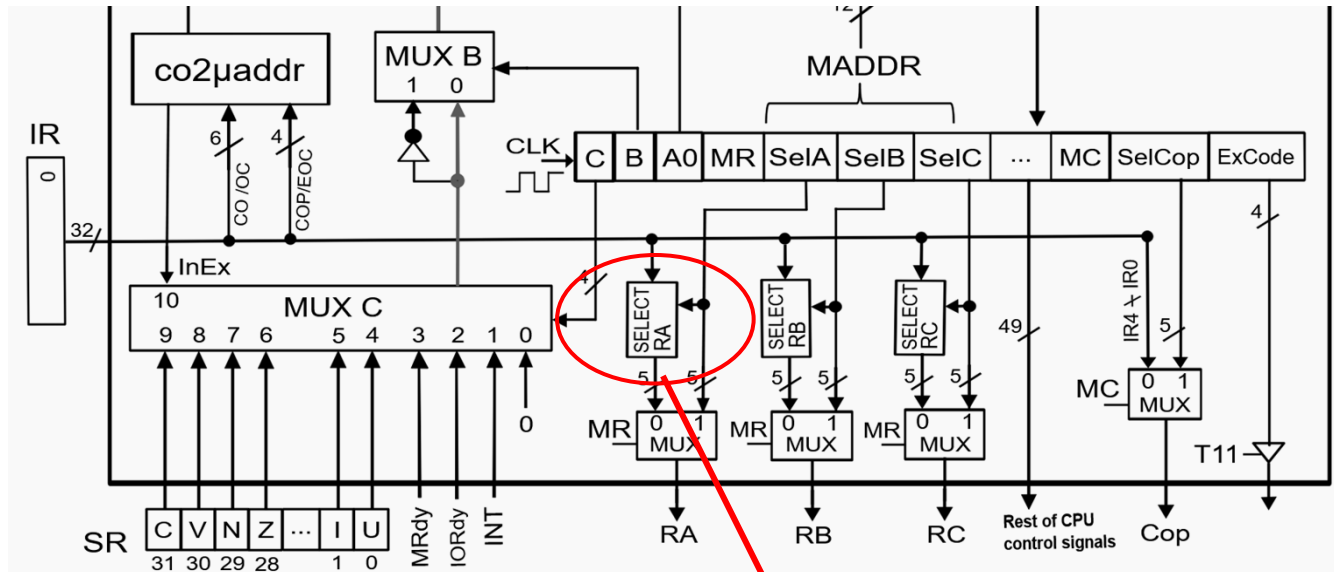
- ▶ (*) If the condition is not satisfied $\rightarrow$ Next μ Address
- ▶ Remaining entries $\rightarrow$ indefinite behaviour

Microinstruction format

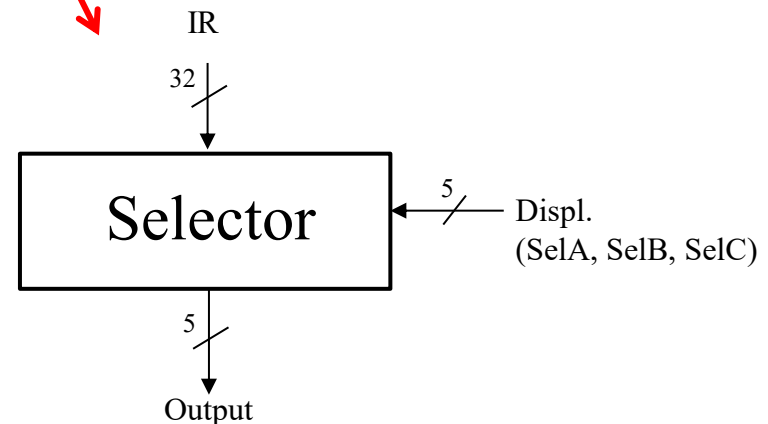


C0 .. C7	Load register
Ta,Td	Tristate buffers to bus
T1..T10	Tristate buffers
M1,M2, M7, MA, MB	Multiplexors
SelP	State register selector
LC	Load in Register File
SE	Sign extensión
Size, Offset	Selector of IR register
BW	Size of memory Access
R, W	Main memory operation
IOR, IOW	I/O operation
INTA	INT selector
I	Enables interuptions
U	User/kernel modes

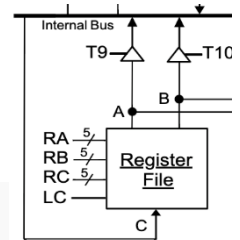
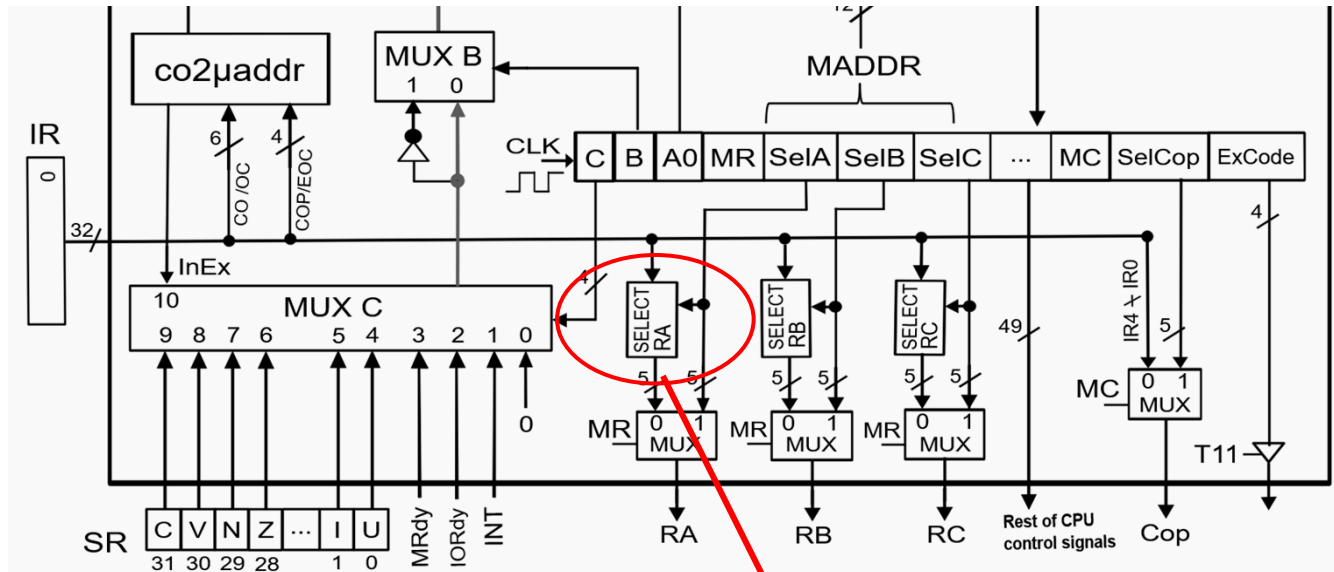
Register file selector



Select 5 bits within 32-bit starting from the position indicated in *Displ* (lower bit)



Register file selector



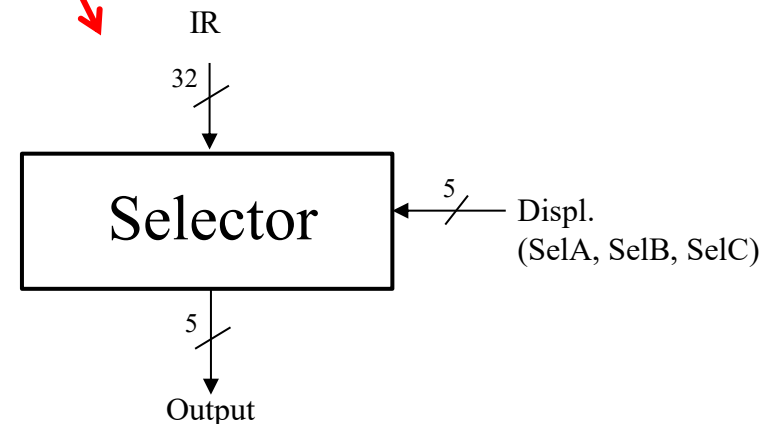
IR: $D_{31}D_{30}D_{29}D_{28}D_{27}D_{26}D_{25} \dots D_4D_3D_2D_1D_0$

If Displ = 11011 $\rightarrow$ Output = $D_{31}D_{30}D_{29}D_{28}D_{27}$

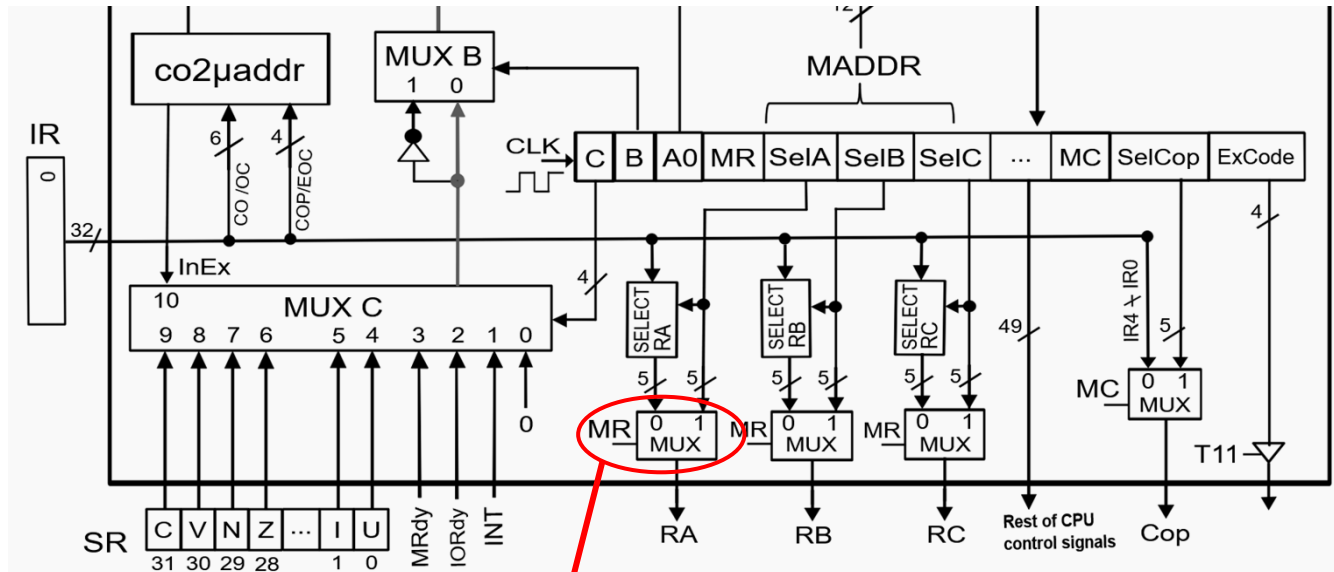
If Displ = 00000 $\rightarrow$ Output = $D_4D_3D_2D_1D_0$

If Displ = 10011 $\rightarrow$ Output = $D_{23}D_{22}D_{21}D_{20}D_{19}$

If Displ = 01011 $\rightarrow$ Output = $D_{15}D_{14}D_{13}D_{12}D_{11}$



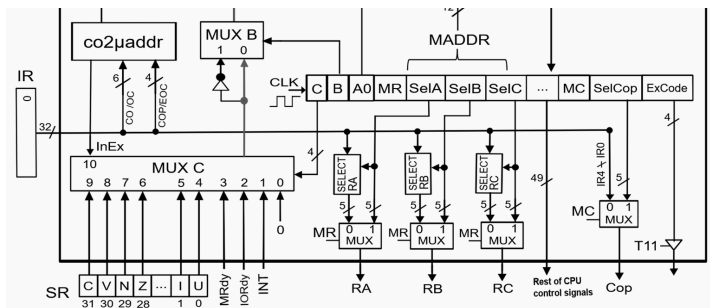
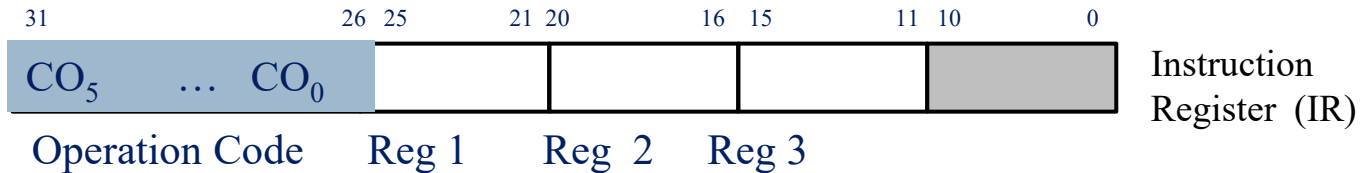
Register file selector



- If MR = 1, RA is obtained directly from the μ Instruction
- If MR = 0, RA is obtained from a field of the instruction (in IR)

Register file selector

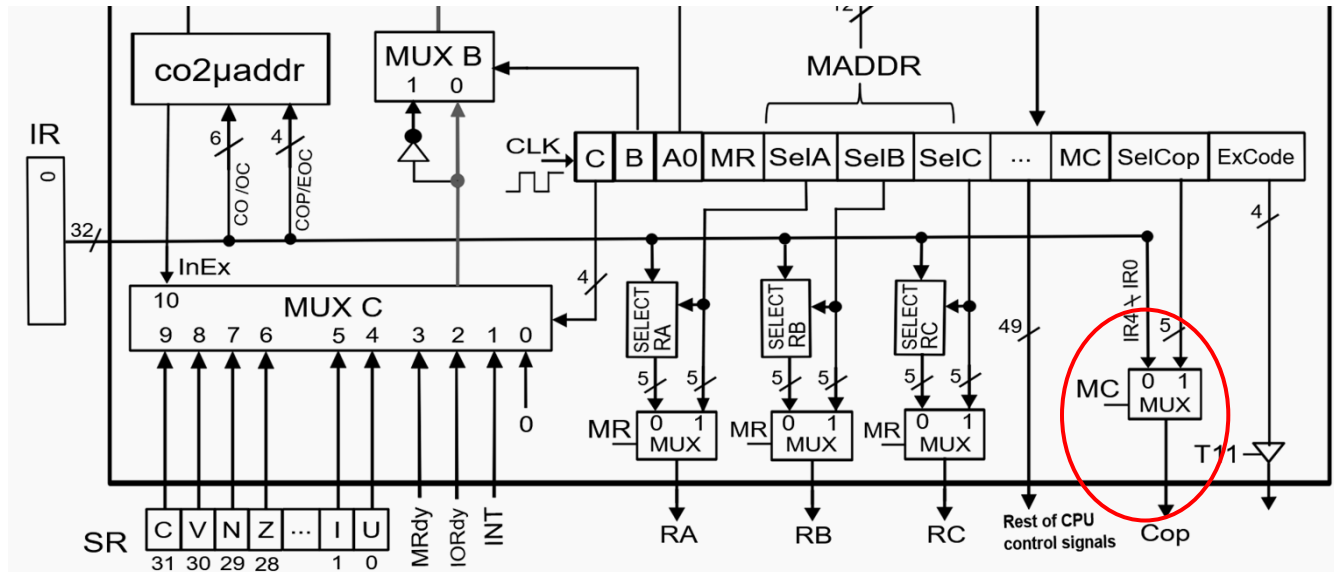
- If the format of an instruction stored in IR is:



$$\text{MR} = 0$$

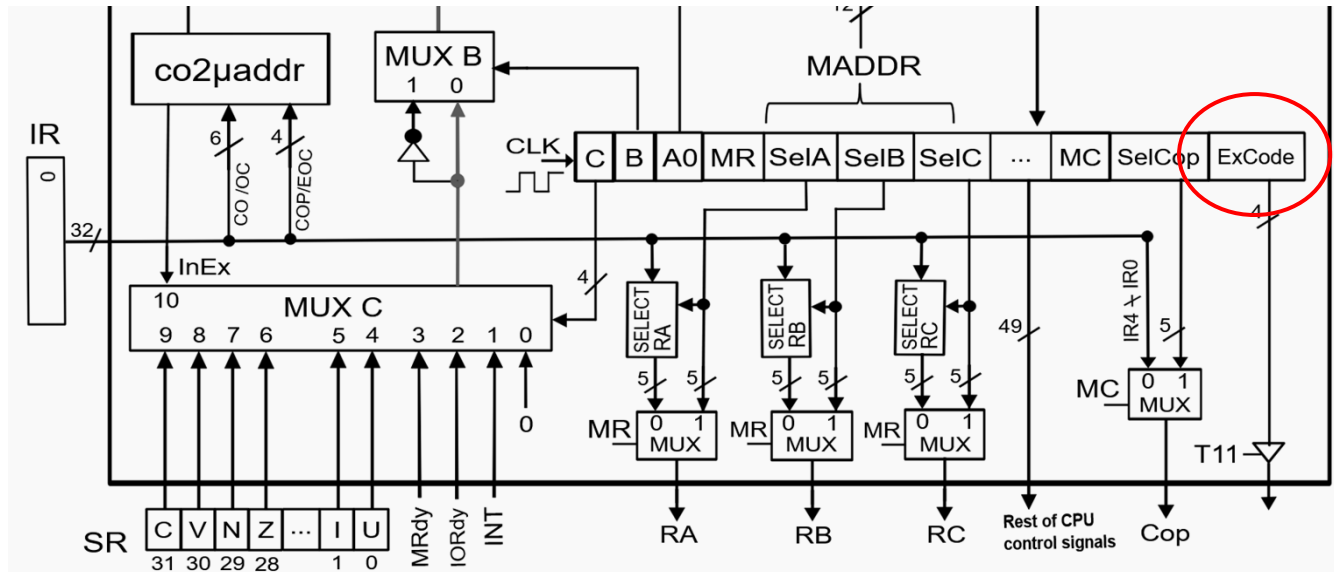
- ▶ If you want to select the field with the Reg 2 in port B of the register file
→ SelB = 10000 (RB is obtained from bits 20...16 of IR)
- ▶ If you want to select the field with the Reg 3 in port A of the register file
→ SelA = 01011 (RA is obtained from bits 15...11 of IR)
- ▶ If you want to select the field with the Reg 1 in port C of the register file
→ SelC = 10101 (RC is obtained from bits 25...21 of IR)

Selection of the ALU operation code



- If **MC = 1**, the operation code of the ALU is obtained directly from the microinstruction (**SelCop**)
- If **MC = 0**, the operation code of the ALU is obtained from the **last four bits** stored in the **instruction register (IR)**

Exception codes



- **ExCode:**
 - Allows to have an immediate value of any 4 bits,
 - Especially useful for generating the interrupt vector to be used when an exception occurs in the instruction.

Contents

1. Computer elements
2. Processor organization
3. The control unit
4. Execution of instructions
5. Control unit design
 - a) Tasks in the design of a control unit
 - b) Microprogram control unit
 - c) Control unit in WepSIM
 - d) Example of a microprogrammed instruction set
6. Execution modes
7. Interrupts
8. Booting a computer
9. Performance and parallelism

Example

► Instruction for microprogramming with WepSIM*:

Instruction	Operation code	Meaning
ADD Rd, Rf1, Rf2	000000	$Rd \leftarrow Rf1 + Rf2$
LI R, value	000001	$R \leftarrow \text{value}$
LW R, addr	000010	$R \leftarrow MP[\text{addr}]$
SW R, addr	000011	$MP[\text{addr}] \leftarrow R$
BEQ Rf1, Rf2, off1	000100	if ($Rf1 == Rf2$) $PC \leftarrow PC + \text{off1}$
J addr	000101	$PC \leftarrow \text{addr}$
HALT	000110	HALT (infinite loop)

* Memory answer in one cycle

Design with the WepSIM control unit

- For each machine instruction:

1. Define the behavior using RTL (register transfer language) for every clock cycle

Instruction

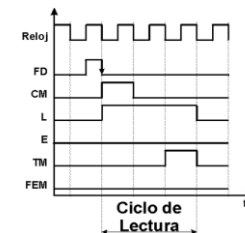
mv R0 R1

Sequence of **elementary operations**

1. IR <- [PC]
2. PC++
3. decode
4. R0 <- R1

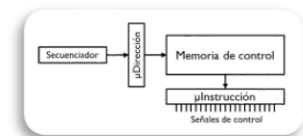
2. Translate the behavior to values of each control signal at each clock cycle

Sequence of **control signals** for each elementary operation



3. Design a circuit that generates the value of each control signal at each clock cycle

Circuit that generates signals:
Microprogrammed control



Microprogrammed instructions

► FETCH

Cycle	Elemental Op.		
0	$MAR \leftarrow PC$		
1	$MBR \leftarrow MP$		
	$PC \leftarrow PC + 4$		
2	$IR \leftarrow MBR$		
3	Decodificación		

Design with the WepSIM control unit

- For each machine instruction:

1. Define the behavior using RTL (register transfer language) for every clock cycle

2. Translate the behavior to values of each control signal at each clock cycle

3. Design a circuit that generates the value of each control signal at each clock cycle

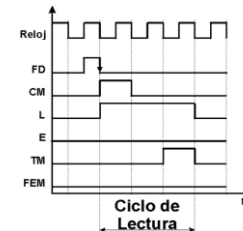
Instruction

mv R0 R1

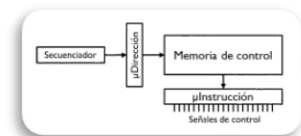
Sequence of **elementary operations**

1. IR <- [PC]
2. PC++
3. decode
4. R0 <- R1

Sequence of **control signals** for each elementary operation



Circuit that generates signals:
Microprogrammed control



Microprogrammed instructions

WepSIM: FETCH example

► FETCH

Cycle	Elemental Op.	Activated signals (rest to 0)	C	B	A0
0	$MAR \leftarrow PC$	T2, C0	0000	0	0
1	$MBR \leftarrow MP$	Ta, R, BW=11, C1, M1	0000	0	0
	$PC \leftarrow PC + 4$	M2, C2	0000	0	0
2	$IR \leftarrow MBR$	T1, C3	0000	0	0
3	Decode		0000	0	1

Design with the WepSIM control unit

- For each machine instruction:

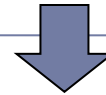
1. Define the behavior using RTL (register transfer language) for every clock cycle

2. Translate the behavior to values of each control signal at each clock cycle

3. Design a circuit that generates the value of each control signal at each clock cycle

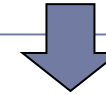
Instruction

mv R0 R1

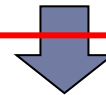
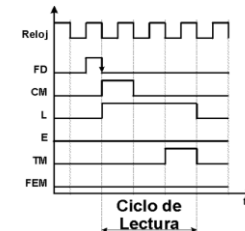


Sequence of **elementary operations**

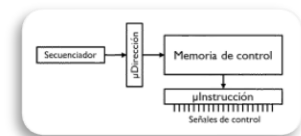
1. IR $\leftarrow$ [PC]
2. PC++
3. decode
4. *R0* $\leftarrow$ *R1*



Sequence of **control signals** for each elementary operation



Circuit that generates signals:
Microprogrammed control



Microprograms in WepSIM

C.	E.O.	Activated signals
0	$MAR \leftarrow PC$	T2, C0
1	$MBR \leftarrow MP,$ $PC \leftarrow PC + 4$	Ta, R, BW=11, C1, M1, M2, C2
2	$IR \leftarrow MBR$	T1, C3
3	Decod.	A0=1, B=0, C=0

Skeleton

<List of microcodes>

<Register section>

<Pseudoinstruccions>

begin

{

fetch: (T2, C0=1),
(Ta, R, BW=11, C1, M1),
(M2, C2, T1, C3),
(A0, B=0, C=0)

}

registers {

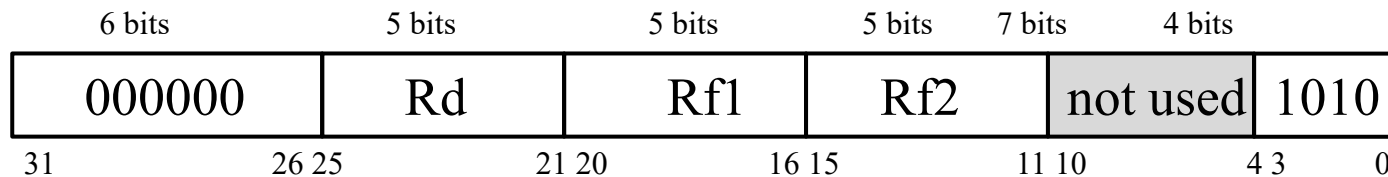
0=(zero, x0), 1=(ra, x1), 2=(sp, x2)(stack_pointer),
3=(gp, x3), 4=(tp, x4), 5=(t0, x5),
6=(t1, x6), 7=(t2, x7), 8=(s0, x8),
9=(s1, x9), 10=(a0, x10), 11=(a1, x11),
12=(a2, x12), 13=(a3, x13), 14=(a4, x14),
15=(a5, x15), 16=(a6, x16), 17=(a7, x17),
18=(s2, x18), 19=(s3, x19), 20=(s4, x20),
21=(s5, x21), 22=(s6, x22), 23=(s7, x23),
24=(s8, x24), 25=(s9, x25), 26=(s10, x26),
27=(s11, x27), 28=(t3, x28), 29=(t4, x29),
30=(t5, x30), 31=(t6, x31)

}

Microprogrammed instructions

► ADD Rd, Rf1, Rf2

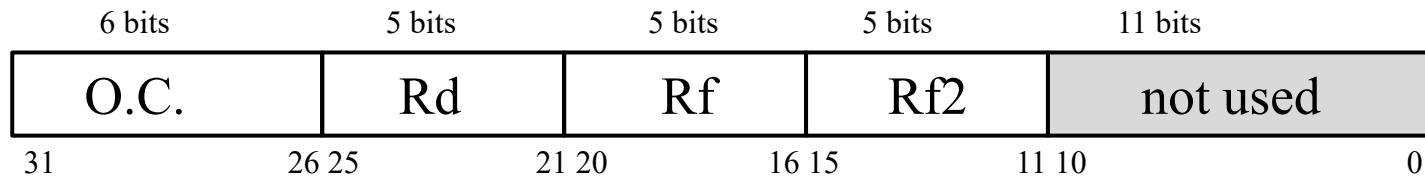
Cycle	Elemental Op.	Activated signals (rest to 0)	C	B	A0
0	$Rd \leftarrow Rf1 + Rf2$	SelA=10000 (16), SelB=01011 (11), MC=0,T6, SelP=11, C7,M7, SelC=10101 (21), LC	0000	1	1



Microprogrammed instructions

► ADD Rd, Rf1, Rf2

Cycle	Elemental Op.	Activated signals (rest to 0)	C	B	A0
0	$Rd \leftarrow Rf1 + Rf2$	SelA=10000 (16), SelB=01011 (11), SelCop=1010, MC=1, SelP=11, C7, M7, T6, SelC=10101 (21), LC	0000	1	1



Defining instructions in WepSIM

WepSIM: ADD examples

```
begin
{
  fetch: (T2, C0=1),
        (T3, R, BW=11, C1, M1),
        (M2, C2, T1, C3),
        (A0, B=0, C=0)
}
```

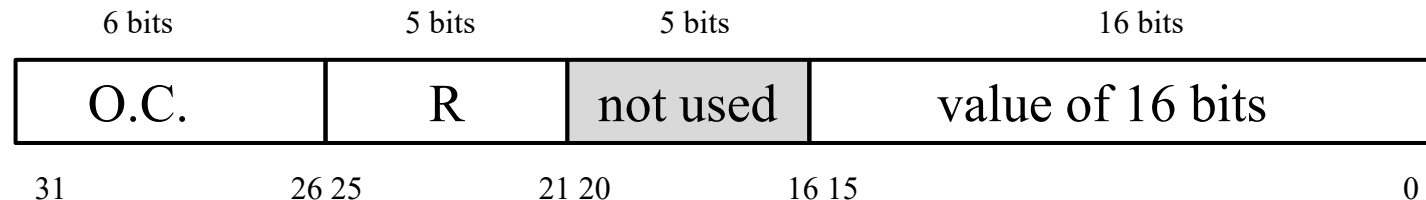
```
add R1, R2, R3 {
  co=100000,
  nwords=1,
  R1=reg(25,21),
  R2=reg(20,16),
  R3=reg(15,11),
  {
    (SelA=01011, SelB=10000,
     SelCop=1010, MC,      SelP=11, M7,C7,
     T6, SelC=10101, LC,   A0=1, B=1, C=0)
  }
}
```

```
registers {
  0=(zero, x0), 1=(ra, x1), 2=(sp, x2)(stack_pointer),
  3=(gp, x3), 4=(tp, x4), 5=(t0, x5),
  6=(t1, x6), 7=(t2, x7), 8=(s0, fp, x8),
  9=(s1, x9), 10=(a0, x10), 11=(a1, x11),
  12=(a2, x12), 13=(a3, x13), 14=(a4, x14),
  15=(a5, x15), 16=(a6, x16), 17=(a7, x17),
  18=(s2, x18), 19=(s3, x19), 20=(s4, x20),
  21=(s5, x21), 22=(s6, x22), 23=(s7, x23),
  24=(s8, x24), 25=(s9, x25), 26=(s10, x26),
  27=(s11, x27), 28=(t3, x28), 29=(t4, x29),
  30=(t5, x30), 31=(t6, x31)
}
```

Microprogrammed instructions

► LI R, value

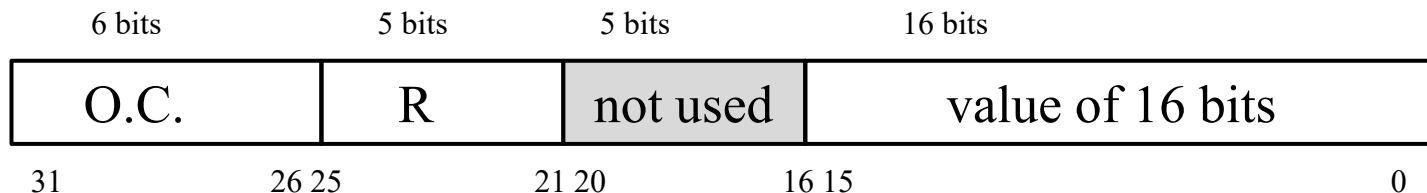
Cycle	Elemental Op.	Activated signals (rest to 0)	C	B	A0
0	$R \leftarrow IR(\text{value})$	LC SelC = 10101 (21) T3, Size = 10000 Offset= 00000 SE=1	0000	1	1



Microprogrammed instructions

- LW R addr # sync memory, 1 clock cycle

Cycle	Elemental Op.	Activated signals (rest to 0)	C	B	A0
0	MAR $\leftarrow$ IR (addr)	T3, C0 Size = 10000, Offset= 00000	0000	0	0
1	MBR $\leftarrow$ MP[MAR]	Ta, R, BW = 11, CI, MI	0000	0	0
2	R $\leftarrow$ MBR	T1, LC, SelC = 10101	0000	1	1

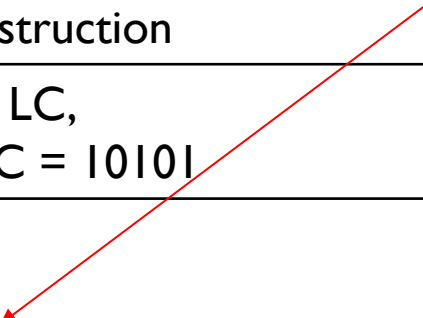


Microprogrammed instructions

- LW R addr # async memory (MRdy=1 for ready)

Ciclo	Op. Elemental	Señales activadas (resto a 0)	C	B	A0
0	MAR $\leftarrow$ IR (addr)	T3, C0 Size = 10000, Offset= 00000	0000	0	0
1	while (!MRdy) MBR $\leftarrow$ MP[MAR]	Ta, R, BW = 11, C1, M1, MADDR= μ Add of this μ instruction	0011	1	0
2	R $\leftarrow$ MBR	T1, LC, SelC = 10101	0000	1	1

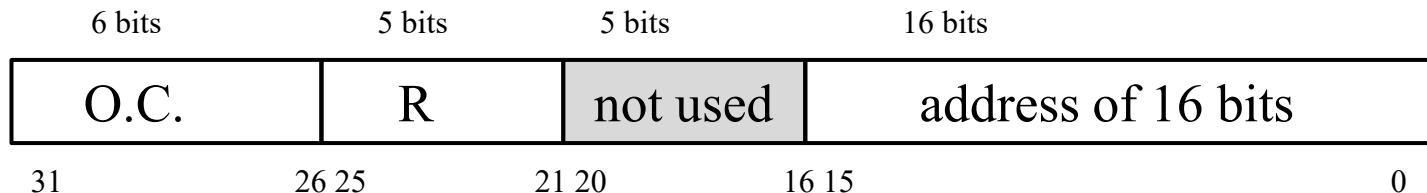
This microinstruction is beening executed while MRdy==0



Microprogrammed instructions

- SW R addr # sync memory, 1 clock cycle

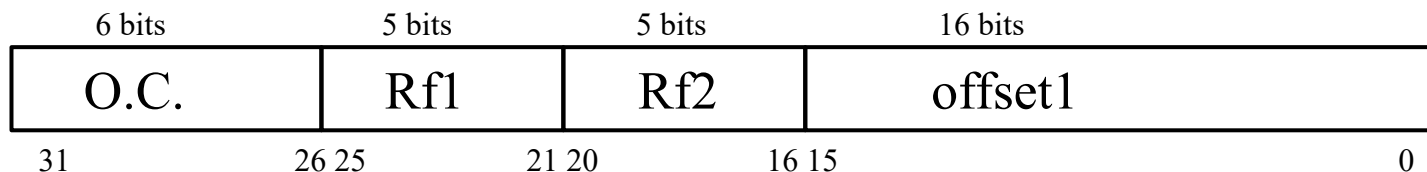
Ciclo	Op. Elemental	Señales activadas (resto a 0)	C	B	A0
0	$MBR \leftarrow R$	T9, C1, SelA=10101	0000	0	0
1	$MAR \leftarrow IR(addr)$	T3, C0, Size = 10000, offset= 00000	0000	0	0
2	$MP[addr] \leftarrow MBR$	Td,Ta, BW = 11,W	0000	1	1



Microprogrammed instructions

► BEQ Rf1, Rf2, offset1

Cycle	Elemental Op.	Activated signals (rest to 0)	C	B	A0
0	Rf1 - Rf2	SelA=10101, SelB=10000, SelCop=1011, MC, C7, M7, SelP=11	0000	0	0
11	If (Z == 0) goto fetch else next	MADDR = 0	0110	1	0
2	RT1 ← PC	T2, C4	0000	0	0
3	RT2 ← IR(offset1)	Size = 10000, Offset = 00000, T3, C5	0000	0	0
4	PC ← RT1 + RT2	MA, MB=01, SelCop=1010, MC, T6, C2	0000	1	1

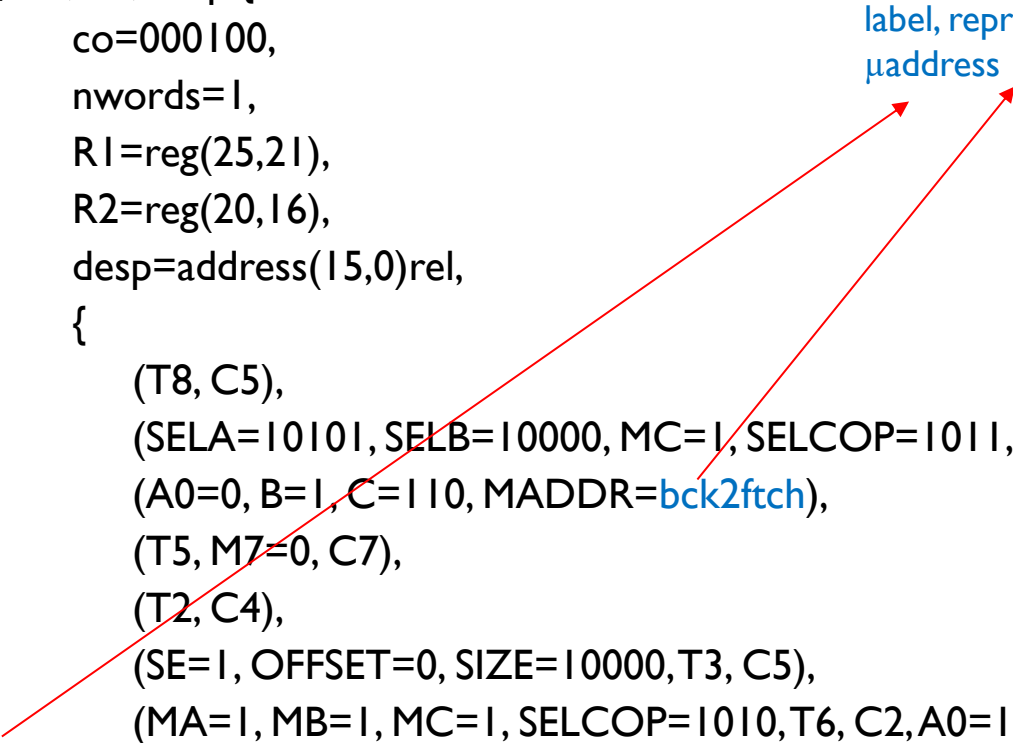


Defining instructions in WepSIM

WepSIM: BEQ example

```
BEQ R1, R2, desp {  
    co=000100,  
    nwords=1,  
    R1=reg(25,21),  
    R2=reg(20,16),  
    desp=address(15,0)rel,  
    {  
        (T8, C5),  
        (SELA=10101, SELB=10000, MC=1, SELCOP=1011, SELP=11, M7, C7),  
        (A0=0, B=1, C=110, MADDR=bck2ftch),  
        (T5, M7=0, C7),  
        (T2, C4),  
        (SE=1, OFFSET=0, SIZE=10000, T3, C5),  
        (MA=1, MB=1, MC=1, SELCOP=1010, T6, C2, A0=1, B=1, C=0),  
bck2ftch: (T5, M7=0, C7),  
        (A0=1, B=1, C=0)  
    }  
}
```

label, represents a μ address



Microprogrammed instructions

► J addr

Cycle	Elemental Op.	Activated signals (rest to 0)	C B A0
0	PC $\leftarrow$ IR (addr)	C2,T3, size = 10000, offset= 00000	0000 1 1



ARCOS Group

uc3m | Universidad **Carlos III** de Madrid

L4: The processor (2)

Computer Structure

Bachelor in Computer Science and Engineering

Bachelor in Applied Mathematics and Computing

Dual Bachelor in Computer Science and Engineering and Business Administration

